Zhuoran Tan[1], Angelos K. Marnerides[1], Christos Anagnostopoulos[1], Shameem Puthiya Parambath[1], and Jeremy Singer[1]

[1]Affiliation not available

January 22, 2024

Posted on 22 Jan 2024 — CC-BY 4.0 — https://doi.org/10.36227/techrxiv.170594149.97651781/v1 — e-Prints posted on TechRxiv are preliminary reports that are not peer reviewed. They should not b...

Advanced Persistent Threats based on Supply Chain Vulnerabilities: Challenges, Solutions and Future Directions

Zhuoran Tan, *Member, IEEE,* Angelos K. Marnerides, *Member, IEEE,* Christos Anagnostopoulos, *Member, IEEE* Shameem Puthiya Parambath, *Member, IEEE,* and Jeremy Singer

***Abstract*—Due to the ever increasing inter-dependency across a variety of diverse software and hardware components in ICT provisioning, supply chain vulnerabilities (SCVs) targeting such dependencies have evolved as a primary choice for malicious actors to initiate stealthy and complex cyber-attacks. The current *modus operandi* within the modern cyber threat spectrum is solely correlated with APTs that have shown to be quite prevalent across diversified attacks underpinning cyberwarfare, cyber terrorism and cybercrime in general. Thus, defense against such events is undoubtedly considered as a high priority on a global scale. Nonetheless, the integration of and dependence on third-party supply chain software and devices located at heterogeneous ICT ecosystems in parallel with the inability of current defense mechanisms to pinpoint nor consider enable a plethora of compromise entry points that consequently amplify APTs. Motivated by these challenges, this survey explores the state-of-the-art to (i) stratify and showcase the properties of supply chain-based APTs, (ii) elaborate on reported risks from such APTs, and, (iii) expand on existing defense methods as proposed until recently. The herein reported study aims to also relate academic research with industry practices having as a greater goal to expose this emerging issue and equip cybersecurity practitioners with the required knowledge for designing next generation APT defense mechanisms.**

***Index Terms*—Advanced Persistent Threats, Supply Chain Attack, Defense Methods, Classification**

List of Abbreviations

AES	Advanced Encryption Standard
ASN	Autonomous System Number
CD	Continuous Delivery/Deployment
CI	Continuous Integration
CSC	Cyber Supply Chain
DES	Data Encryption Standard
DF	Domain Fronting
DSA	Digital Signature Algorithm
MBR	Master Boot Record
RSA	Rivest Shamir Adleman
SCC	Satellite-Based Communication Channel
SCVs	Supply Chain Vulnerabilities
SMB	Server Message Block
SP	System Profiling
SSL	Secure Sockets Layer

Zhuoran Tan, Christos Anagnostopoulos, Jeremy Singer and Shameem Puthiya Parambath are with the School of Computing Science, University of Glasgow, Glasgow, Scotland, G12 8RZ. Angelos K. Marnerides is with the KIOS Centre of Excellence and the Department of Electrical & Computer Engineering, University of Cyprus, Nicosia, Cyprus. Corresponding Author: Zhuoran Tan, e-mail: z.tan.1@research.gla.ac.uk

I. Introduction

According to the latest Mandiant report [1], the majority of APTs related to ransomware attacks were orchestrated via vulnerabilities in third-party supply chain software. Moreover, the Worldwide Application Security Project (OWASP) whitepaper emphasizes that Supply Chain Vulnerabilities (SCVs) attributed to third-party software dependencies have been ranked as the top 10 vulnerabilities in large language models [2]. Furthermore, the European Union Agency for Cybersecurity (ENISA) whitepaper [3] reported that twenty-four large-scale cyber-attacks during the period of 2020-2021 were due to SCVs. Among them, around 50% of the attacks were attributed to APT groups. Since the exposure of the SolarWinds supply chain attack in 2020 [4], there is a broader tendency to exploit SCVs. Due to the stealthy and massive impact of the supply chain [5], the supply chain-based APT has exacerbated the security situation. Hence, developing efficient and powerful defense products and their deployments has become an urgent priority.

APTs arising from SCVs are not studied extensively. The majority of existing studies focus primarily on general APTs [6]–[10], with limited consideration for supply chain exploitation. Notably, the related survey [11] on APTs fails to mention supply chain attacks. On the other hand, works that focus on supply chain attacks lack in-depth analysis of advanced exploitation techniques [5], [12]–[15]. It is important to recognize that most supply chain attacks are non-targeted, distinguishing them from targeted APTs. To the best of our knowledge, there is no comprehensive solution developed to defend against this type of APT. To address this gap, we conduct this survey and analyze the state-of-the-art.

We conduct a comprehensive literature review, encompassing both academic and grey literature [16]. The academic literature review aims to identify state-of-the-art defense methods against supply chain attacks and APT exploitation. The majority of them are from top-tier journals or conferences. In contrast, the review of grey literature directs its attention towards firsthand occurrences and practical industrial scenarios, thereby providing invaluable information. This review includes white papers, reports from reputable research labs, blogs with detailed vulnerability exploitation processes, and books published by academic institutions. These materials, based on real-world attack scenarios, provide up-to-date information and valuable insights from domain experts. Our main objective

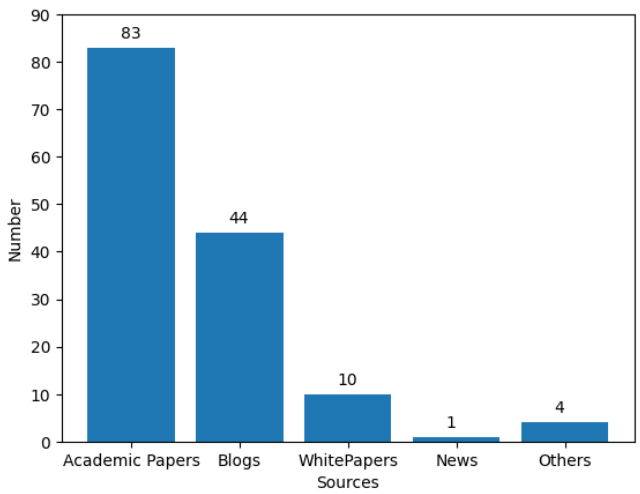

Fig. 1: Literature Statistics.

is to identify the core techniques used in supply chain-based APTs to conceal their behaviors and offer guidance for efficient and optimized defense safeguards. The distribution of different source materials used for this survey is given in Figure 1. Academic research and blogs account for over half of the total citations, signifying the latest studies explored in this work. Additionally, we include white papers primarily sourced from top cybersecurity companies, e.g., Mandiant [1], LogRhythm [2], CrowdStrike [3]. The main contributions of this survey include:

- A comprehensive literature review regarding supply chain attacks and potential APT attacks based on SCVs. We point out the difference between the two groups of attacks. Furthermore, we firstly dive into the specific techniques applied by them via investigating authentic APTs based on SCVs, which has been not systematically explored before.
- Taxonomy of supply chain threats, combining insights from industry, academia, and government standards. The scope of threats covers software, hardware, dependencies, and etc. By further divided with separate components and stages, we cover most of the potential attack vectors disclosed so far, which is the the most comprehensive enumeration to best of our knowledge.
- Taxonomy of the most recent defense methodologies against APT attacks and supply chain attacks. The defense methodologies have been classified into three main categories including; (i) detection methods, (ii) censoring methods, and (iii) proactive methods, in which there exist additional triage.
- A novel universal threat model to demonstrate APTs based on supply chain vulnerabilities. The threat model unifies the hardware and software supply chain and includes core components involved in a complete supply chain attack.
- Insights into applied techniques in supply chain attacks by adopting graph-based and statistical analysis. We utilize graph-based analysis to visualize strongly overlapped techniques applied by multiple APT groups. The statistical analysis can help directly identify the stages of APT where adopting disparate techniques.
- Outlining future research directions regarding the detection of APTs based on SCVs. We denote the lack of research focusing on this specific type of APT by delving into the coverage of current defence technologies. We also points out the drawbacks and limitations of those defence methods to further instruct more advanced detection research for APT based on SCVs.

The remainder of this paper is structured as follows: Section II provides the definition of APT and how it can be illustrated under Cyber Kill Chain with involved common techniques. Section III deals with the components involved in supply chains and the triage regarding supply chain compromises from different standards. In section IV, we compare APTs with traditional supply chain attacks, carry out the statistic analysis for applied techniques, propose the threat model, and provide two case studies including software and hardware. Section V discusses the defence methodologies for those attacks and further classify different approaches. Section VI presents the discussion on current challenges and further research opportunities to efficiently detect those attacks. Finally, Section VII summarizes and concludes the paper.

II. Advanced Persistent Threats (APTs)

In this section, we begin with the concept of the Cyber Kill Chain (CKC) to illustrate the generic steps of any attack vector. Subsequently we discuss APTs by synthesizing the tailored stages of the Cyber Kill Chain. Finally, we delve into a more detailed description of the common exploited techniques and stages during APT attacks.

A. Cyber Kill Chain (CKC)

The concept of the 'Cyber Kill Chain' originated as a military concept, introduced by Lockheed Martin [4] in 2011. The CKC points at a series of steps that trace stages of a cyberattack from early reconnaissance stages to the exfiltration of data. The typical stages include target identification, weapon delivery, attack initiation, and ultimately, target destruction or exfiltration, which also motivates the understanding of APTs. Over the years, this concept has been extensively applied in information security and forms the basis for the widely recognized MITRE matrix. In addition, the content of it has been enriched to other variants, like the Unified Kill Chain (UKC) [17]. The UKC offers significant improvements over these scope limitations of the CKC and the time-agnostic nature of the tactics in MITRE matrix, which has been considered as the most comprehensive and popular version for distinguishing APT stages.

Despite certain common processes shared by APTs, the techniques they employ vary significantly. In complex systems,

[1] https://www.mandiant.com/
[2] https://logrhythm.com/
[3] https://www.crowdstrike.com/
[4] https://www.lockheedmartin.com/en-us/capabilities/cyber/cyber-kill-chain.html

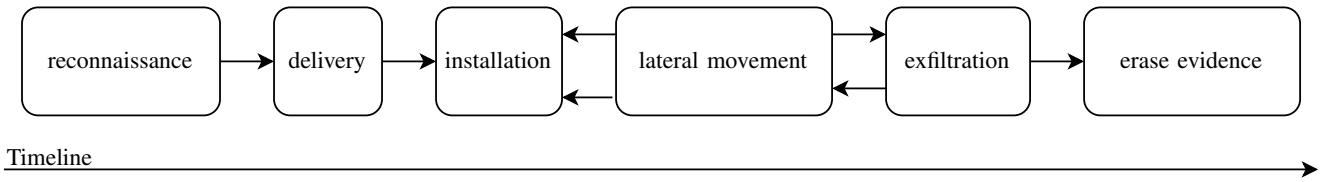

Fig. 2: APT Stages.

this diversity of techniques tends to expose more attack surfaces, making the system susceptible to potential exploitation. A notable example of this is the exploitation of the Cyber Supply Chain (CSC), which has become increasingly prevalent for APT actors [5], [12], [18]. The use of the Kill Chain model and the understanding of its evolution have significantly contributed to the identification and detection of APTs. However, it is essential to acknowledge the constant evolution of APT techniques and be vigilant in adopting comprehensive and adaptive approaches to defend against these sophisticated threats, especially when dealing with intricate systems and potential CSC vulnerabilities.

B. APT Decomposition

An APT is a sophisticated and targeted long-term cyberattack orchestrated by well-resourced and highly skilled adversaries [19]. Originating from military contexts, the APT concept has expanded into the realm of daily IT security society [19]. These attacks were initially directed at nation-states and related entities but have since broadened their targets to encompass private and organizational institutions as well [20]. Defending against APT attacks has become increasingly challenging due to their unique characteristics, including long-term latency, uncertain strategies, and strong evasion capabilities [21]. APT attacks exhibit salient attributes such as slow and low movement or volume, high complexity, and a multi-stage nature [6]–[8], [17], [20]. Consequently, cyber-analysts often face limited insights and a scarcity of data when investigating such attacks. Figure 2 illustrates the typical stages of an APT attack, which include:

In order to explicitly illustrate the meaning and context of each stage, we primarily refer to tactics defined in the MITRE Adversarial Tactics, Techniques, and Common Knowledge (MITRE ATT&CK) framework[5]. To the best of our knowledge, MITRE provides a more comprehensive definition of techniques, tactics, and procedures (TTPs) for various attack types. Additionally, it is a widely used framework for defense purposes in the industry, including Security Operations Centers (SOCs), Incident Response, Attack Simulation, and more.

Reconnaissance (RE): An APT attack is characterized as a targeted attack, where the attackers aim to collect sufficient information about the targeted organization's network, employees, and topology. The ultimate goal is to identify numerous potential attack vectors to establish a foothold for further infiltration. Two common methodologies employed during an APT campaign are active and passive information gathering. They differ based on whether there is direct access to the target network or not. Active gathering points to an interactive network scan and probe to acquire information regarding the system or network of the target. In this category, the attacker often scans network ports, identifies vulnerabilities, fingerprints operating systems, and verifies services, potentially generating logs or alerts. In contrast, passive gathering does not leave an obvious activity trace. It involves the exploitation of the open-source intelligence (OSINT) framework[6], designed to collect information from public sources. Additionally, listening to network traffic, sniffing, and phishing are part of this group. As a reference, MITRE adopts the tactic of reconnaissance for this stage. During this stage, attackers may prepare malicious tools or software, corresponding to the "Resource Development" tactic in MITRE ATT&CK. This preparation potentially includes acquiring or developing malware, domains, and other resources that support subsequent tactics, such as initial access, command and control, and defense evasion. These resources play a crucial role in the attacker's ability to execute and sustain their attack campaign effectively.

Delivery: This stage involves sending the attackers' weapons to the target system. These weapons consist of remote access malware or malicious code [19], also known as payloads, which can be used in later stages to control the compromised target or conduct further exploitation. The delivery of these malicious payloads typically occurs through commonly used mediums, such as email attachments, compromised websites, or USB drives. During this stage, attackers must convert the information and vulnerabilities gathered in the previous phase of information gathering into a malicious payload. This conversion process is essential for achieving the initial foothold within the target system, referred to as 'initial access' in the MITRE framework.

Installation: In diverse delivery scenarios, the installation process of malicious payloads occurs following actions such as clicking malicious links in email attachments or visiting compromised websites. In more sophisticated cases, adversaries may employ trigger mechanisms[7] based on rare events, a typical sequence of events, or scheduling tasks [22]. The SolarWinds attack [4] serves as an example where a certain concealment period was used as the launching mechanism to evade detection by defense systems. This stage aligns with tactics specified in MITRE, encompassing execution, persistence, and privilege escalation exploitation. The first tactic, execution, involves executing the malicious code to gain unauthorized access (UA) to the target system. Subsequently, the persistence tactic comes into play, allowing the download of malicious tools or software, such as malware, to maintain

[5]https://attack.mitre.org/

[6]https://osintframework.com/

[7]https://attack.mitre.org/techniques/T1546/

persistent control even after system reboots or the implementation of security measures. In many cases, the attacker endeavors to establish a hidden backdoor (HB) within the target organization's system to achieve persistence objectives.

Lateral Movement: After successfully infecting the device of one end user within the target institution, the attackers engage in lateral movement, which involves expanding their reach by gaining control over other machines or devices that may hold valuable structural information or sensitive data. Due to the restrictions imposed by access control lists or least privilege policies, the initially compromised account often lacks sufficient privileges. As a result, privilege escalation becomes a crucial aspect, enabling the attackers to elevate their access level by exploiting other vulnerabilities within the internal network. Furthermore, to facilitate lateral movement, the attackers may steal or obtain valid credentials, effectively performing a second reconnaissance, which corresponds to the 'discovery' tactic in the MITRE framework. With access to additional machines secured, the goal of lateral movement is firmly established, allowing the attackers to maneuver further within the target network. This phase is critical for expanding the scope of the attack and gaining access to high-value assets and sensitive information within the organization.

Exfiltration: Various APT attacks have distinct objectives, but they share a common goal: data exfiltration (DE). In this context, adversaries aim to collect sensitive or valuable information from compromised systems or network resources. The data they target may include financial data, document theft (DT), intellectual property theft (IPT), personal (ID) data, and credit card information, among others. Once the attackers have gathered the desired information, their next step involves transferring it to controlled external servers. This transfer is accomplished through the command and control (C2) method, which entails communication channels used to issue commands, exfiltrate data, and receive further instructions from the attackers. Through these channels, the stolen data is sent to the designated external servers, providing the attackers with access to the exfiltrated information for their malicious purposes.

Erase Evidence: To evade detection and tracking, advanced attackers employ various methods during the post-exploitation phase. They may choose to delete logs (DL) or remove malware artifacts (RMA) generated by their activities or simply uninstall the malware when it is no longer needed. These actions can be performed manually or automated by incorporating self-destruction mechanisms (SDM) into malicious tools [23], [24]. This stage highlights the significance of proactive and real-time detection in the early stages of APTs to mitigate potential damage. The removal of logs and malware artifacts is a primary reason why tracking APT activities after the conclusion of the full cycle becomes exceptionally challenging. The manipulations carried out during this stage may overlap with the 'impact' tactic defined in the MITRE framework, which involves actions aimed at disrupting or damaging the target environment. Consequently, containing the impact to this stage becomes essential to minimizing the overall damage caused by the APT attack.

Although we provide a series of universal stages to describe APTs, there are slight differences in practice. These differences can arise from the time spent and the difficulties encountered at each stage. The reconnaissance stage, for example, can extend over months to collect enough information for the target. The enumeration of attack vectors is a time-consuming task, typically taking more than half of the entire penetration process. The installation stage depends on how the trigger mechanism is configured by the adversary. If the execution is not triggered, the malicious code can remain silent for an extended period. During other stages, such as lateral movement, the available exploitations can be diverse, and it takes an uncertain amount of time for attackers to exploit the proper ones to reach their goals.

Another variation lies in the potential order of stages; an attack may repeat some stages or reverse their order during exploitation. A direct example is the download of second-stage malware, which is generally a more powerful tool used for in-depth reconnaissance inside the target system. This allows attackers to find additional vulnerabilities to compromise user accounts with higher privileges in the internal network.

In conclusion, APTs can span from weeks to months, which explains why detection systems may not always locate APTs in time. Part of the reason is that the majority of current systems lack the ability to correlate data over a long period to identify malicious behaviors, primarily due to computation and storage limitations. Apart from that, the dataset regarding long-term APT activity is also not available yet, which causes the few research regarding detection of APT within a long time span.

C. Common Techniques in APTs

The various stages typically encompass a wide array of techniques and sub-techniques, and there is no strict delineation dictating which specific techniques should correspond to a particular stage. Often, a single technique can potentially be repurposed across multiple stages. This flexibility informs our decision to detail common Tactics, Techniques, and Procedures (TTPs) predicated on the investigation of exposed APT attacks, rather than constraining specific techniques within stage descriptions, as conducted in [20].

Hidden Backdoor In the effort to maintain consistent unauthorized access to compromised systems, hackers can employ intrinsic access points, typically constituted by segments of code embedded within software or systems. These 'hidden backdoors' (HB) facilitate unauthorized access for attackers, enabling them to circumvent security protocols and remain undetected over extended periods. The HB can be deployed for the initial compromise, establishment of persistence, as well as for lateral movement or further exploitation [24]–[27].

Command and Control This refers to the strategy attackers employ to sustain communication and exert control over compromised systems, with the manipulated hosts commonly referred to as bots or zombies [28]. This technique can be utilized to issue commands, receive updates, and more. It also has the capacity to serve as a conduit for installing payloads (IPS) from external servers. This methodology can be implemented in stages, such as the initial compromise, establishment of persistence, reconnaissance, and lateral movement.

Self-Updating Mechanisms (SUM) This mechanism highlights an update process that is free from manual manipula-

tion. While certain settings or triggering conditions, such as downloading or installation, are required, the update process largely occurs automatically. This technique, often implicated in the spread of malware, ordinarily entails the substitution of legitimate updates or versions with malicious ones. It has been observed at various stages of cyber attacks, including the initial intrusion stage, the establishment of persistence step, and the evasion of detection procedure [28].

Hijack Update (HU) The hijack update attack constitutes one method that can be used to compromise the update mechanism (CUM). This tactic is characterized by the interception and manipulation of software updates for nefarious purposes [29]. Malefactors can employ this approach at various stages of a cyber attack, including the initial compromise, privilege escalation, persistence, and lateral movement.

Custom Protocols (CPS) We introduce the term CPS as an amalgamated term for a collection of interconnected techniques. This framework generally alludes to the strategies that adversaries can manipulate to facilitate undetected message exchange or payload delivery for Data Exfiltration (DE) or delivery stages. Subsumed within this classification are procedures such as encrypted Hypertext Transfer Protocol (HTTP), Traffic (EHT) [30], Custom DNS protocol (CDP), and Hypertext Transfer Protocol Secure (HTTPS) Protocol (HSP) [31]. In terms of liaising with the C2 server, malicious activities, or command execution (CE), can be effectuated through highly deceptive custom protocols, which convincingly masquerade as their legitimate counterparts.

Watering Hole (WH) This method bears semblance to a trap wherein frequently visited websites or platforms by targets are compromised [32]. The objective of attackers utilizing this technique is to exploit the trust relationship between visitors and these compromised websites. The full exploitation cycle, typically comprising four stages—selection of targeted websites, compromise of these websites, exploitation of trust, and subsequent infection or exploitation—can often be observed during both reconnaissance and post-exploitation stages.

Hijack Dynamic Link Library (DLL): This refers to the process of substituting or altering a legitimate Dynamic Link Library (DLL) with a malicious one. DLLs are the implementation of shared libraries in the Windows system. Miscreants have exploited this to gain higher privileges for escalation and to propagate to other systems within a compromised network [33]. In some unique instances, DLL hijacking forms part of the operations for vulnerability exploitation in the system or software at the initial access stage [34].

Code Signature Spoofing (CSS): This is a technique employed to tamper with the digital signature of software or code, rendering it seemingly legitimate or trustworthy. It can facilitate the distribution of malicious software undetected and evade security mechanisms reliant on code integrity verification. Attackers can leverage this method during initial access [35] to deliver malicious code to target systems. Moreover, this technique can successfully conceal malicious code for persistence without arousing suspicion.

Obfuscate Activities (OA): This pertains to techniques utilized by adversaries to mask malicious activities, enhancing their evasion from detection [36], [37]. Sub-procedures can encompass methodologies such as modifying system timestamps (MST) and code obfuscation, among others. Perpetrators can use this for reconnaissance or initial access, and for lateral movement and privilege escalation stages. Additionally, this process plays a supportive role in malware delivery and execution. There also exists a corresponding method known as anti-forensic techniques (AFT) that serves to either avoid or impede forensic analysis of the attack behavior. This is primarily deployed in latter stages of a cyber attack to erase any incriminating evidence.

Remote Command Execution (RCE): This emphasizes the capability of an adversary to execute commands on a target system or network remotely. It enables the attacker to seize control of the target system, execute arbitrary commands to extract information, or accomplish malignant objectives. It can form part of the initial access stage following the exploitation of software vulnerabilities or potentially weakly protected customer-facing web interfaces. Furthermore, attackers can utilize this technique for lateral movement, such as executing commands on additional systems to broaden the impact. Lastly, this method can also serve roles in maintaining persistence, privilege escalation, and data exfiltration.

Network Packet Interception (NPI): This refers to the analysis of network packets. Depending on the depth of inspection for the context, it can be divided into two sub-techniques: Deep Packet Interception (DPI) and Stateful Packet Interception (SPI) [38]. The latter focuses on tracking the state of network connections, such as Transmission Control Protocol (TCP) streams, Internet Control Message Protocol (ICMP) messages, and so on. During the reconnaissance stage, it can be employed in collecting information about the target network. For initial access, DPI can aid in identifying potential credentials or session token information. Moreover, by monitoring and analyzing internal network packets, it can assist in locating other devices for lateral movement. Also, it can support the data exfiltration stage by retrieving valuable or sensitive data from network traffic.

Wi-Fi Evil Twin (WET): In the context of wireless attacks, WET has emerged as both a fundamental and prevalent method used to dupe users into connecting to a rogue access point that impersonates a legitimate Wi-Fi network [39]. Typically deployed as a trap during the initial access stage, it can also be used for credential theft. Furthermore, this technique serves as a means designed for network reconnaissance.

III. Supply Chain Compromise

The supply chain comprises a sequence of distinct products or services delineated across multiple institutions or vendors. It encompasses mechanisms of product or service delivery. Due to its diverse nature, it poses challenges to auditing, investigation, and integration. Each intermediate stage or element within the supply chain could potentially be targeted by adversaries before reaching the final consumer. As a result, supply chain-based APT attacks have seen an uptick in recent years and have become highly impactful threats. Such attacks typically possess excellent levels of concealment and persistence. To understand the occurrence of supply chain

attacks, it is essential to comprehend the components involved in the supply chain, followed by the triage of attacks targeting those components.

A. Supply Chains

The concept of supply chains is expansive, encompassing the combination of organizations, individuals, resources, and technology involved in the production, distribution, and delivery of services to end users [3], [40]. In [41], the authors pointed out the entire life cycle of the supply chain, from the sources of products and services to integration through design, development, manufacturing, and delivery. In this project, we emphasize more on the specific subset of the CSC, which pertains to the digital network of organizations, processes, and systems involved in the creation, distribution, and management of digital products, services, and components [18].

According to the National Institute of Standards and Technology (NIST) [42] framework, the ecosystem of CSC relationships can be simplified into two interconnected groups: technology and non-technology elements. These groups only intersect at the organization level but have distinct suppliers and buyers depending on their affiliation with technology. The NIST cybersecurity framework, proposed by the U.S. Department of Commerce, was developed to assist businesses of all sizes in understanding, managing, and mitigating their cybersecurity risks, as well as to safeguard their networks and data. It also provides guidance on managing cybersecurity within the supply chain. In the more detailed Cybersecurity Framework version 1.1 Manufacturing Profile [43], during the identify stage, the category of supply chain management is emphasized to provide clear guidance on how to navigate the associated risks. Taking into account the different categories identified in NIST's framework and other definitions, we have extracted core components to facilitate a comprehensive discussion on the CSC.

a) Software and Hardware Suppliers: An Internet-facing company invariably relies on various software and hardware. Such a company is often not capable of building everything from scratch, necessitating the incorporation of third-party devices, such as customer relationship management (CRM) systems, to manage customers. Specific companies specialize in providing these tools, systems, or hardware to meet unique needs [4], [44], [45]. Additionally, open-source software communities [16] play a pivotal role for most software suppliers. Due to their popular usage among potentially millions of end users, these suppliers become primary targets for adversaries keen on exploiting the supply chain to instigate attacks. The NIST cybersecurity framework [42] specifies in subcategory ID.BE-1 for the business environment the need to identify external upstream and downstream supply channels, which include a variety of suppliers. The supply chain-specific subcategory ID.SC-3 also underscores the enforcement of control requirements for suppliers and third-party providers.

b) Development and Integration: The entire process of software and hardware development and integration involves intricate divisions of labor, cooperative efforts, and well-defined procedures [13], [22]. It encompasses the intricate design and integration of hardware and software components. Additionally, activities such as coding, testing, and quality assurance are integral parts of the development process to ensure the achievement of desired functionality [14]. To facilitate these activities, various development and auxiliary tools are indispensable to support the entire life cycle of the software and hardware development process. Based on the exploration, the development platform has also been proven to be one of the main targets for adversaries [18], [22].

c) Distribution and Deployment: This component pertains to the delivery and deployment of software, services, or hardware to end-users, encompassing crucial activities such as installation, packaging, and activation. Throughout this process, each stage is vulnerable to the injection of malicious code (IMC). For instance, during the delivery stage, attackers may tamper with or manipulate the configuration [22]. Similarly, in the packaging stage, various components, such as code contributors and package maintainers, pose potential risks of compromise or uploading malicious code [14]. As a result, it is essential to diligently address security concerns at each stage of the delivery and deployment process to safeguard against the injection of malicious elements. Meanwhile, ensuring the integrity of the software, services, or hardware provided to end-users is also significant.

d) Service Providers: The services available in the cyber world have grown increasingly diverse in recent times, encompassing management, general IT, storage, cloud, and computation services. These services have played a crucial role in accelerating technological advancements within organizations. However, as integral components of the supply chain, they contribute to the increased complexity of networks and structures, presenting a broader attack surface for malicious actors. Addressing and securing all potential attack vectors has become an urgent and immensely challenging task. This specific concern has been recognized and incorporated as part of ID.SC-3 with high impact in relevant cybersecurity frameworks [42].

e) Supply Chain Security: Supply chain security revolves around the actions and practices taken to achieve the aim of security protection. In particular, researchers in [22] have identified the key goals of supply chain cybersecurity, which include confidentiality, integrity, authenticity, and exclusivity of components. Some other studies [16], [46] have also highlighted aspects such as integrity and authenticity. The introduced four objectives encompass all digital products and services throughout the supply chain. To accomplish these security goals, activities such as assessment and risk management, as well as the implementation of security controls, are crucial for associated suppliers, integrated software, and hardware components [43]. These measures help mitigate vulnerabilities and protect against potential threats at various stages of the supply chain, ensuring the overall security and integrity of the products and services delivered to end-users.

f) Threats and Risks: The threats in the context of CSC refer to potential attacks that can target the supply chain. According to [41], supply chain attacks are incidents where attackers exploit vulnerabilities within the product or service supply network of their targets. During the development or

build stage, adversaries may compromise the build environment (CBE) to gain unauthorized access or control over the environment used for building software or firmware. This compromise allows the attacker to inject malicious code into the software or hardware [16], leading to the infection of downstream components in the supply chain. Another common threat is the exploitation of zero-day vulnerabilities in software (ZDS) [47], [48]. Zero-day vulnerabilities refer to unknown vulnerabilities in software that are not yet patched or mitigated by the software vendor. Adversaries exploit these vulnerabilities to gain unauthorized access or compromise the software, potentially leading to significant security breaches and damage.

g) Compliance and Regulations: Organizations operating within the CSC must adhere to industry standards and best practices to effectively mitigate risks. As suggested in [41], the usage of EU and national laws can help prevent supply chain attacks. Another study in [49] focused on reproducible builds (RBs) for software supply chain security, highlighting challenges and proposing possible improvement directions. The SLSA framework [8], a supply chain-specific framework, provides guidelines for best practices in defending against and mitigating supply chain-related threats. This framework also offers approaches to measure compliance with executive order standards within the Secure Software Development Framework (SSDF) established by NIST. Additionally, [43] mentions the subcategory ID.SC-4, which emphasizes the review and assessment of compliance with contractual obligations, indicating the importance of compliance and regulation across various stakeholders.

B. Classification of Supply Chain Compromises

The classification of supply chain attacks is essential for deploying efficient defense approaches and detection methods. Triage methodologies have been explored by government institutions such as ENISA [3] and NIST [43]. Academic studies, such as [13] and [22], have conducted extensive enumerations of attack vectors for software and hardware, providing valuable insights for detailed classification. In the realm of industrial work, two widely recognized frameworks, the MITRE and the SLSA frameworks [16], offer analyses of categorical supply chain threats. The MITRE framework emphasizes the objects that supply chain attacks target, while the SLSA framework highlights divisions based on the supply chain life-cycle, encompassing the source, build, and package stages from producer to customer. The SLSA framework provides more detailed threat categories related to the software supply chain. By employing these classification methods, organizations can better understand and categorize the various threats they may encounter in their supply chain, enabling them to implement more targeted and effective security measures to safeguard against potential attacks. In the following context, we provide a comprehensive introduction to the two main classification methods introduced in industrial frameworks.

[8]https://slsa.dev/spec/v1.0/about

1) MITRE ATT&CK Framework: The MITRE ATT&CK framework is a comprehensive knowledge base that provides valuable information about the tactics and techniques used by adversaries during cyberattacks. It was developed and is maintained by the MITRE Corporation, a not-for-profit organization that focuses on research and development in various fields, including cybersecurity. In the subset of the supply chain compromise technique, three subcategories are given to encompass all possible types of supply chain attacks.

a) Software Dependencies and Development Tools: The development of software invariably relies on external libraries or tools aimed at accelerating the entire process and ensuring quality. However, this opens up opportunities for adversaries to compromise the developed software through their dependencies, including widely used open-source or closed-source projects [5], [13], [50]. Moreover, these popular projects may be extensively distributed to a wide consumer base, potentially affecting millions of individuals or institutions. Notable breaches have occurred in top-referenced projects [50]–[55], which are widely utilized in a vast number of software applications. For exploitation, attackers often introduce an innocuous dependency and later update the dependency to incorporate malicious behaviors. Consequently, any software that relies on the infected versions or updates can be recursively infected. Detecting such malicious activity is often challenging for general intrusion detection systems without thorough checking for dependent libraries.

b) Software Supply Chain: Within a software supply chain (SSC), adversaries may carry out manipulations on application software in several ways before it reaches the end-user. These modifications can encompass altering the source code, manipulating the update or distribution mechanism, or substituting official releases with modified versions. One of the most notable instances of such an attack is the SolarWinds supply chain attack, where a trojanized version of the Orion software was implemented [56], [57]. Some academic work has studied SSC compromise and provided valuable insights.

The study in [16] delves into the taxonomy of attacks on Open-Source Software (OSS) supply chains. The researchers illustrated potential threats across all stages of the OSS supply chain, from code contribution to package distribution. Through a systematic literature review, they conducted tree-testing on 94 real-world incidents and compiled 107 distinct attack vectors. Employing both qualitative and quantitative analyses, the researchers assessed the validity and comprehensiveness of their investigation outcomes using statistical methods and interviews with 17 domain experts. They also evaluated the efficacy and cost-effectiveness of current safeguard solutions and their usage rates. For future work, the researchers plan to investigate the prevalence of identified countermeasures in open-source projects and update the taxonomy with new attack vectors. Additionally, they intend to conduct more empirical studies to validate the taxonomy and assess its usefulness. Another aspect of their future agenda includes developing automated tools for detecting and preventing such attacks.

Similarly, a qualitative study conducted in [58] delves into the security challenges of the Open Source Software (OSS) chain. By conducting 25 in-depth interviews with experts, this

research identified inherent challenges, such as the presence of plausibly unvetted contributors, obligations to access and mitigate the impact of vulnerabilities in external components, and the necessity for more dedicated teams or tools to better audit included components. By highlighting these challenges, the study underscores the importance of contributing to the health and security of the overall software ecosystem, which can yield substantial benefits. The study employed a rigorous experimental design, utilizing a semi-structured interview approach and ensuring the recruitment of participants with many years of experience. Additionally, a stage for member checking was included, involving sharing results with interviewees to ensure the validity and accuracy of interpretations. However, as a qualitative study, the findings are based on the experiences and perceptions of a convenience sample of participants. To address this potential limitation, the researchers plan to explore the best approaches for different company types to support or engage in open-source practices. Moreover, they intend to examine how companies can enhance their development processes concerning open-source components to bolster overall security and efficiency.

The work in [49] explored the challenges of Reproducible Builds (RBs) for software supply chain security using a qualitative method. This research involved 24 semi-structured expert interviews with participants from the Reproducible-Builds.org project. The results revealed challenges that primarily revolve around a lack of awareness and understanding, limited supportive tooling and infrastructure, and difficulties in achieving RBs due to differences in build environments. The study underscores the importance of RBs and provides recommendations on how to make RBs a commonplace practice. However, potential limitations include the generalizability of the findings, biases, and the lack of diversity in the participant selection, which may not fully represent other geographical and professional groups. Additionally, a comprehensive analysis of the technical aspects of RBs was not provided. The researchers' future plans involve conducting more empirical studies, developing tools and frameworks to automatically create RBs, and refining technical standards and best practices for RBs.

Indeed, various works have delved into different aspects of software supply chain security, each focusing on specific ecosystems and procedures. For instance, [13] hones in on the Java ecosystem, while [14] concentrates on package managers for interpreted languages. Additionally, [59] explores the security of pre-trained models stored on model hubs.

While these works address specific supply chain contexts, they collectively highlight common procedures that traverse the entire software supply chain, from source code development to the distribution of software artifacts. In our comprehensive discussion of the threat model, we thoroughly examine potential attack vectors, as detailed in Part III of Section IV, aiming to encompass diverse scenarios and challenges associated with software supply chain security. By analyzing these attack vectors, we can better comprehend the vulnerabilities and risks that may arise at various stages of the software supply chain, enabling us to devise effective countermeasures and enhance the overall security and integrity of the supply chain. In contrast to the software supply chain (SSC), the hardware supply chain operates quite differently for each case.

c) Hardware Supply Chain: In contrast to software, the hardware supply chain (HSC) involves physical components, intricate manufacturing processes, and unique logistical considerations. A prevalent method of exploitation in HSC is to insert backdoors into the targets' network through modifications in integrated hardware or firmware, granting malicious actors unauthorized privileges to control the system and conduct further malicious activities. We present some academic outcomes targeting unique hardware infrastructures.

In [46], the authors studied a potential approach to inject malicious code into software guard extensions (SGX), which serves as a method to preserve the confidentiality and integrity of applications. SGX operates as a trusted execution environment (TEE) in Intel processors [9]. The injection attack could occur between the time of building the software binary and its signing process. The core component that needs to be signed is called an enclave. In the attack scenario, the attacker utilized malware to receive enclave binary code as input and generated the signing material by assuming pre-obtained access to a machine where the signing material is generated. The malware intercepted and suspended the signing process, enabling the injection of malicious code. Consequently, a rogue enclave could be endorsed by a valid signature. This attack can corrupt the SGX binary enclave and extract sensitive information through data exfiltration. The specific exploitation methodology depends on the malicious actor's knowledge of a particular enclave functionality. To defend against this, one possible solution involves centralized mitigation by creating a secure channel between the build and signing servers. Another solution is a smart contract deployed on a blockchain infrastructure. However, both mitigation methods have drawbacks. The centralized approach may not be suitable for large enclaves due to SGX memory constraints, while the distributed solution may require additional infrastructure. The next plan involves evaluating and comparing the proposed mitigation methods.

Another example is mentioned in [60]. The authors proposed a novel attack vector within the electronics supply chain, marking the first comprehensive study of computer-aided design (CAD)-based attacks. They explored how malicious CAD tools can introduce vulnerabilities into a system-on-chip (SoC). Various exposed attack vectors exist at every stage of the design process, including high-level synthesis and logic synthesis. For instance, in the case of malicious high-level synthesis, unused clock cycles can be maliciously employed to implement a backdoor with a rare trigger mechanism, making it challenging to detect. These introduced hardware Trojans can launch stealthy malicious modifications in a design and lead to data breaches. However, the authors did not consider attacks by a malicious operating system (OS) that can modify CAD binaries, and potential malware that modifies CAD tools to launch attacks. Additionally, they did not provide solutions for these security concerns. For future research directions, it is suggested to use this work as a reference to design more

[9]https://www.intel.co.uk/content/www/uk/en/homepage.html

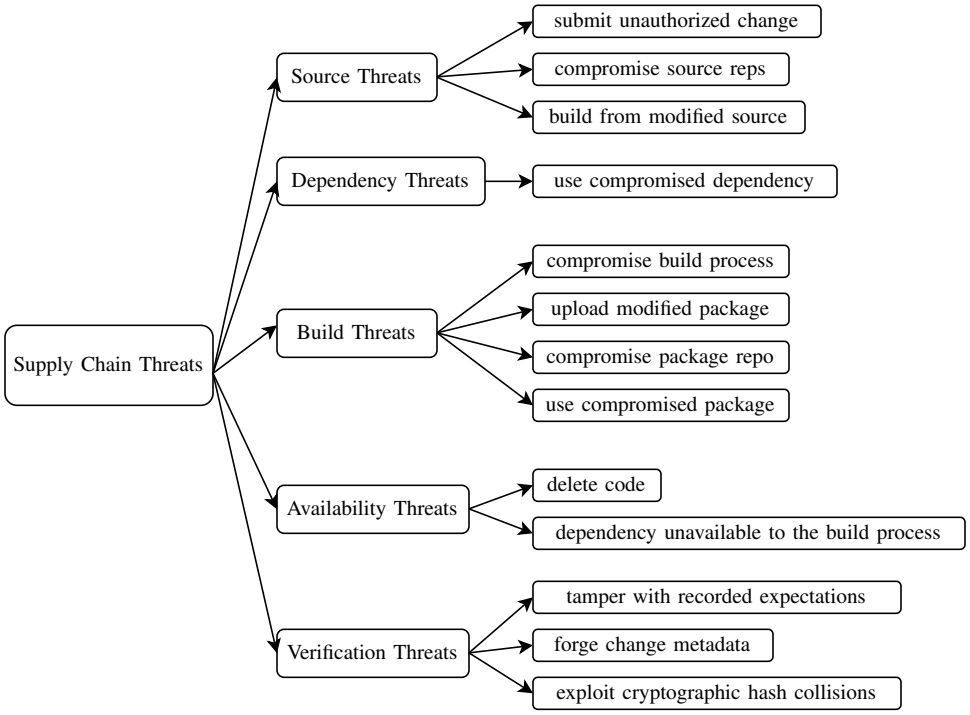

Fig. 3: Classification of Supply Chain Threats based on SLSA

attacks and develop countermeasures to mitigate potential threats.

Despite variations in hardware structures among different cases, the hardware supply chain (HSC) shares similarities with software supply chain threats at each stage of its life cycle [22]. Throughout the HSC, potential vulnerabilities exist, such as modifications or insertions during the development stage, the theft of intellectual property and sensitive information during the design stages, and other potential attack vectors. Furthermore, the hardware supply chain has seen the emergence of novel exploitation techniques, including hidden communication channels employing XOR-encrypted steganographic (stego) techniques [61]. These techniques aim to surreptitiously conceal unauthorized communications, presenting challenges in terms of detection and prevention.

In light of these evolving threats and shared vulnerabilities, robust security measures are imperative for the hardware supply chain. Implementing physical security practices, conducting continuous integrity checks, and closely monitoring for any suspicious activities or modifications in the hardware components or firmware are essential steps to safeguard against potential attacks in the hardware supply chain.

2) Supply Chain Levels for Software Artifacts: Supply Chain Levels for Software Artifacts (SLSA) is a specialized framework that offers best practices for supply chain security, targeting both producers and customers. To the best of our knowledge, it is the most comprehensive industrial framework specifically dedicated to addressing supply chain threats. The framework emphasizes two primary security goals: integrity and availability. SLSA classifies threats into three main categories with two additional categories, as depicted in Figure 3. This classification takes into account the various components involved in the supply chain life cycle. These components comprise two subjects: the producer and the consumer, as well as three key processes: source, build, and package. Additionally, the framework considers the dependencies associated with the build process as an auxiliary part.

a) Source Threats: The source threats within the SLSA framework primarily pertain to risks occurring before the build stage, indicating a higher likelihood of insider threats. This category encompasses three specific threats, namely "Submit Unauthorized Change", "Compromise Source Repository", and "Build from Modified Source".

The "Submit Unauthorized Change" threat involves the introduction of unauthorized changes by adversaries through the official source control management platform or interface. Such attacks do not necessarily require a higher level of privilege. In most cases, contributors with malicious intentions act as the primary attackers [14], [59]. Implementing proper checks on modified parts can help prevent malicious contributions to some extent.

The "Compromise Source Repository" threat targets the compromise of package managers, which possess administrative privileges to control interfaces and repositories. After gaining sufficient access privileges, adversaries can directly introduce malicious code and perform other unauthorized manipulations. Preventive measures include implementing multi-factor authentication (MFA) for the platform to ensure the appropriate privileges for official users [14], [55].

The "Build from Modified Source" threat represents one of the most commonly triggered methods in the supply chain [5], [13], [16], [58]. Within this framework, the authors identify five possible unmatched locations of modified sources: unofficial forks of code, branches or tags, build steps, parameters,

and modified versions of code. An "unofficial fork" refers to forks that are not built from official authority and may contain unauthorized changes introduced by attackers. The work by Yi [62] provides comprehensive research regarding forked projects. To mitigate this threat, the verifier side can match the provenance's source location. The same mitigation method can be applied to prevent the following three subsets. Addressing the modified version of code is a primary focus of research in supply chain-based attacks. By comparing different versions [5], the malicious parts can be identified and patched or fixed in updated versions. A general mitigation method also involves updating the related software or code to the latest version.

b) Dependencies Threats: The SLSA framework identifies the "Use Compromised Dependency" threat as a critical vector wherein adversaries introduce malicious software or libraries during the build process. Although these dependencies are often employed to provide additional functionalities and expedite development, their benign nature is not guaranteed. SLSA further categorizes this threat into two distinct types: "Compromised Build Dependency" and "Run-time Dependency". The term "Compromised Build Dependency" refers to the injection of malicious code into the software required for building the artifact. This necessary software is typically part of the compiled source code. On the other hand, "Run-time Dependency" pertains to the software needed to execute the artifact, often linked dynamically to run as a binary version.

To mitigate this threat, SLSA recommends verifying the provenance data of dependencies before fetching them. This process ensures the authenticity and integrity of the components used in the software supply chain, helping prevent the introduction of compromised elements into the final product.

c) Build Threats: The final main category of build threats encompasses attacks that can target any stage from the build process to the final consumer. This category is further subdivided into four subcategories: Compromise Build Process, Upload Modified Package, Compromise Package Repo, and Use Compromised Package. To provide a more detailed classification of these threats, additional divisions are made based on specific aspects of supply chain security, such as build platform, project owner, and documentation. This results in a total of four levels of threat tracks.

At Build Level One (L1), the focus is on identifying and addressing mistakes and documentation issues, utilizing provenance to manage threats effectively. Build Level Two (L2) centers on tampering that occurs after the build process, where signed provenance generated by a hosted platform can help address challenges. The highest level, Build Level Three (L3), emphasizes tampering during the build process itself, with the recommended approach to mitigate such threats being to harden the build platform. A detailed illustration of this fine division can be found in Figure 4, along with further explanations and discussions on different technologies aligned with the various build levels.

The approach of compromising the build process primarily focuses on altering the build output, either by tampering with the build process itself or by injecting false information into the provenance. During the build stage, identified as Level 3 (L3), adversaries could potentially compromise other builds, steal cryptographic secrets, or poison the build cache. Among these, compromising other builds refers to tampering with benign builds operating concurrently or subsequently. This typically involves the substitution of source files in a benign build with those from a malicious one. However, the provenance of the benign build may falsely indicate that benign source files were used. This type of attack can arise from the potential overlap of different builds operating on the same machine. Cryptographic secret theft pertains to illicit access to cryptographic secrets related to the build platform of the provenance signature. This can result in false provenance and misdirection of data source verification. Similarly, poisoning the build cache can have the same adverse effect. Post-build stage or at Level 2 (L2), there are two primary sub-threats: forging values of the provenance and compromising the project owner. The former can have a similar impact as the theft of cryptographic secrets for the provenance signature, while the latter can provide adversaries with sufficient privileges.

The threat of uploading a modified package includes four subcategories: building with an untrusted CI/CD infrastructure, uploading a package without provenance, tampering with the artifact post CI/CD process, and tampering with provenance. Two of these threats, namely, uploading a package without provenance and tampering with the artifact post CI/CD, fall under Level 1 (L1) threats, pertaining to malicious modifications of the artifact related to the package itself. Specifically, uploading a malicious package could be integrated into the build software in the absence of adequate verification and provenance, including hash checks of the package. Tampering with provenance typically occurs after an unauthorized build, where the provenance is altered to bypass security checks, classified as a Level 2 (L2) threat. The remaining threat, building with an untrusted CI/CD infrastructure, refers to the use of unofficial CI/CD pipelines not correctly constructed. The key method to counteract this type of attack is through the verification of builder provenance.

The threat of "compromising package repo" is direct and can result in the repository's inability to serve artifacts or provenance. The final identified threat is the use of compromised packages, aiming to deceive the user into utilizing unintended packages. A prevalent method of exploitation is typosquatting [14], [16], [59], a technique also frequently employed in phishing websites or emails [63], [64]. Typosquatting capitalizes on human visual errors by replacing letters in the package name with visually similar but malicious alternatives.

In addition to the primary categories of threats, the remaining threats have been split into two main groups: availability threats and verification threats. Availability threats include the deletion of code and unavailable dependencies, both of which undermine the availability of the supply chain. Currently, there are no known effective prevention methods for these threats. Verification threats, on the other hand, pertain to those that compromise the integrity of the supply chain. In these instances, adversaries may modify original information without detection or potentially bypass existing controls in place.

In conclusion, five distinct categories of threats have been

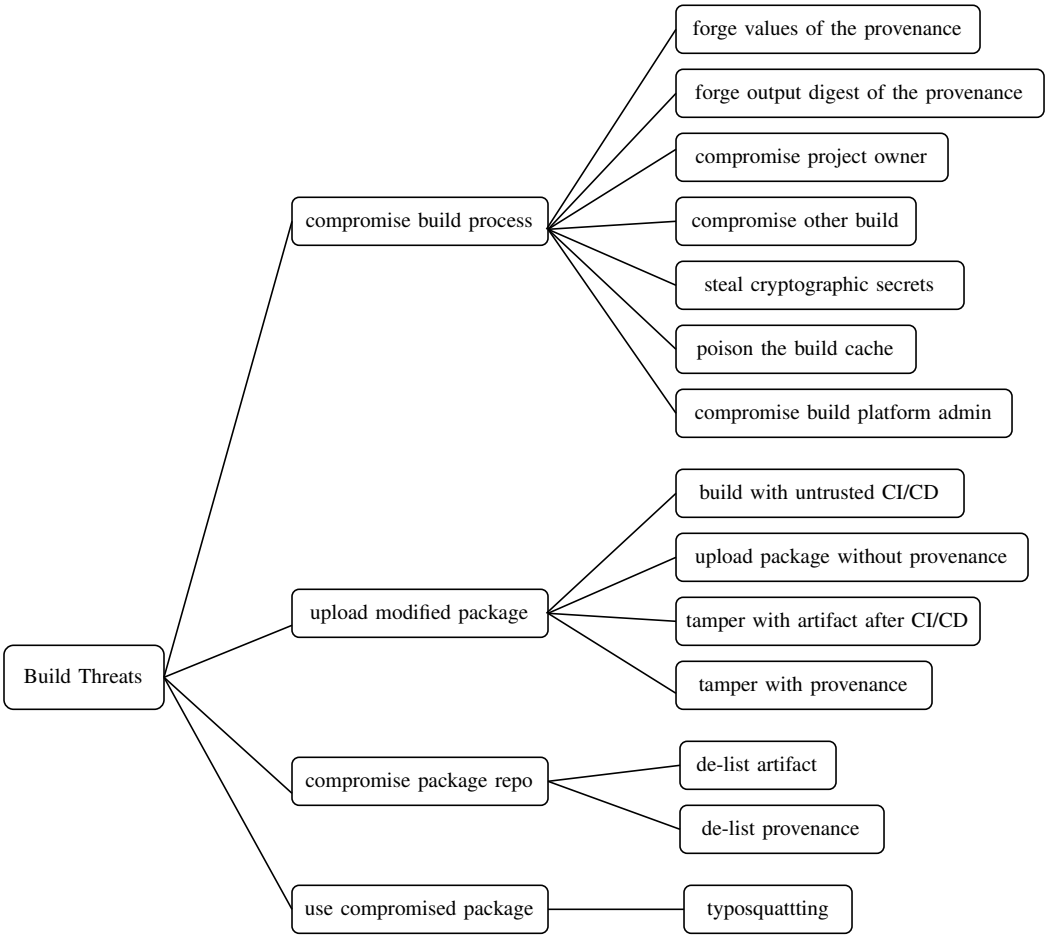

Fig. 4: Classification of Build Threats

identified. These categories can guide developers in implementing secure development strategies and direct defenders to more efficient deployment and optimization of security checks. However, when these threats are integrated into APTs, the potential damages and subsequent challenges for defense need to be critically evaluated.

IV. Supply Chain-based APT Attacks

A. APTs vs. Traditional Supply Chain Attacks

An attack that qualifies as an APT possesses distinct characteristics. According to [20], three primary attributes differentiate advanced attacks from common attacks: the effort taken for prevention, the requirement for high adaptation, and the exhibition of novelty in attack variants. This conceptual framework can be used to distinguish threats against the supply chain from other APTs. A Supply Chain Attack (SCA) is an integral component of supply chain-based APTs. However, typically, an SCA does not specify targets. The software or hardware targeted [52]–[54] relies absolutely on its value or popularity, gauged by its number of downloads. More downloads increase the magnitude of potential downstream impacts. Using the example of OSS supply chain attacks, this type of attack spans stages from code contribution to package distribution [16], effectively encompassing the entire development cycle of software. Nevertheless, the exploitation of software primarily serves as an initial stage for APTs, creating unauthorized access or establishing a foothold for subsequent exploitations. In the following parts, we demonstrate specific techniques inside APT stages and conclude with a threat model and attack vectors from real-world attack scenarios.

B. Malicious Techniques

Numerous APTs have alarmingly managed to remain undetected for extended periods. Investigating the mechanisms these threats employ to evade existing defense protocols and successfully achieve their objectives is a critically significant aspect of this study. Understanding the technologies utilized by sophisticated threat agents is essential, as it informs effective and prioritized defense implementation. This understanding could reveal limitations within existing defense solutions and inform more robust deployments.

In this section, we conduct extensive statistical analysis on 14 exposed instances of long-term APT activity in recent years, as outlined in Tables I and II. Most technology names have been manually substituted with corresponding ID names listed in the MITRE Enterprise Techniques Table[10]. Our investigation dichotomizes the techniques used in the initial stages and the methods deployed for concealed communication. We

[10]https://attack.mitre.org/tactics/enterprise/

Name	Techniques	Goals
XDSpy [65], [66]	T1203, T1137.001, T1587.001, T1595, T1005, T1589	RE, DT
Modified Elephant [67]	T1587.001, T1587.004, T1056.001, T1583.006, T1078.004, T1020, T1219	Surveillance
Cozy Bear [68], [69]	T1087.002, T1110.004, T1539, T1047, T1098.001,T1137.001, T1550.002, T1550.003, T1190 ,T1053.005	Espionage
TajMahal [24], [70]	T1542.001, T1055.001, T1115, T1114, T1113, T1608.001, T1588.005, T1040, T1056.001, SP, T1125, SUM, SDM	Infiltration, Espionage
Fancy Bear [71], [72]	WH, T1566, ZDS, T1056.001, T1113, DT, RE	Espionage
Lazarus Group [73], [74]	WH, T1566, ZDS , RCE, T1565, T1056.001, T1113	Financial Gain, Espionage
Equation Group [25]	ZDS, T1014, T1542.003, NPI, RE, DE, HB, CE, IPS	Espionage, Surveillance
Comment Crew [75]	T1566, WH, ZDS, T1211, HB, T1219	Espionage
Hidden Lynx [76]	T1566, WH, ZDS , T1219, T1056.001, T1113	Espionage, Economic and Political Gain
Winnti Group [77]	T1566, WH, T1195 , HB, T1056.001, T1219	IPT, Espionage, Financial Gain
DarkHotel [78]	WH, T1566, T1056.001, T1219	Espionage, IPT, DE
OceanLotus [26]	WH, T1566, HB, T1078, T1056.001, T1219	Espionage, UA
Turla Group [27]	HU, HB, WH, T1112, T1014, HD, DT, T1056.001, T1113	IPT, Espionage, Intelligence Gathering
Supermicro Hardware [44], [79]	T1200, T1546.008	Espionage, DE

TABLE I: Undetected APT Attacks Investigation Part I

The technique names that start with T follow MITRE ATT&CK framework

focus primarily on communication-related technologies as preferred means of stealth, based on the assumption that APTs regularly communicate with external servers.

Moreover, we examine all techniques apparent in their activities to attain comprehensive insights. The inclusion of APT objectives serves to inform the creation of the threat model in subsequent sections and to differentiate from common supply chain attacks. Furthermore, we preferentially probe into APT cases that exploited SCVs, aiming to gauge the degree of risk for exploitable tendencies in APT strategies.

1) Initial Stage: The initial stage of an attack refers to the point at which adversaries gain unauthorized access to a target's system. This stage aligns with the 'initial access' tactic delineated in the MITRE ATT&CK framework and constitutes the second stage, or 'delivery stage', in an APT. Enhanced depth and accuracy for the compiled information are achieved by incorporating reconnaissance and resource development techniques. During our analysis, we have attributed a high frequency to techniques appearing more than twice within the total APT samples.

From the statistical analysis in Chart 5 for techniques enumerated in the 'Initial Stage' column of Table II, we find that T1592 and T1589 have the highest frequency, each occurring five times. T1592 and T1589 refer, respectively, to gathering information about the victim's host and identity, two common methods embodied within the Resource Enhancement (RE) tactic. This information-gathering strategy encompasses all facets related to the host (including hardware, software, firmware, and configuration) and personal data.

Indeed, the gathered personal information often primarily serves phishing attempts by accumulating email and credential data. However, due to limitations in the extracted information, we are unable to assign specific sub-techniques for these two methods. Two other techniques, T1590 and T1566, are noted to have a frequency of four. T1590 focuses on collecting data of the victim's network, which could include administrative information pertaining to topology and operations. This technique, along with T1592 and T1589, falls under the banner of the RE tactic, which aims to identify vulnerable vectors in the system.

Concurrently, technique T1566 symbolizes a phishing attack. Within this classification, there are three subdivisions: spear-phishing attachment, spear-phishing link, and spear-phishing via service. Spear-phishing represents a more targeted form of phishing, crafted specifically for an individual, a company, or an industry. It is pertinent to mention that phishing emails have been identified as a principal attack mechanism, contributing to nearly 91% of successful cyber-attacks [80].

The subsequent techniques, T1595 and T1591, each having a frequency of three, warrant consideration. T1595 indicates active scanning, a method that attempts to probe the victim's infrastructure via network traffic. This tactic is particularly effective in identifying open ports and often leaves a trace during the scanning process. An inherent component of this method includes vulnerability scanning, which, by its deployment, can unveil additional attack vectors. Of particular note, antiquated service or system versions may be detected, potentially exposing unpatched vulnerabilities.

T1591 aligns with T1592 and T1589 in its focus on information gathering, specifically catering to organizational information. In consistency with the collected data, techniques falling under the reconnaissance tactic have been recorded twenty times, the highest among all tactics. This is somewhat analogous to exploits, as both tactics attempt to target and exploit system vulnerabilities. This inference aligns with the findings of the Mandiant 2023 Report [1], which discloses that exploits, at 32%, occupy the highest percentage of initial infection vectors in targeted attacks.

When combining the two subclasses of T1566, both having an occurrence of 2, we find that phishing attempts have a cumulative frequency of eight, ranking second. This aligns with the results from the same Mandiant report [1], which designates phishing as the second most utilized vector for initial infection in observed intrusions. It is specified to account for 22% of intrusions where the initial infection vector was identified.

It is important to note that within the collected APT attack scenarios, exploitation based on SCVs only occurs twice, as evidenced in two subtypes of T1195: T1195.003 and T1195.002. These respectively correspond to compromises in the software and hardware supply chain. These findings coincide with the data from [1], wherein the proportion of supply chain compromise within initial compromises amounts to only 0.2%. In essence, supply chain compromise does not constitute a primary concern for adversarial actors as of now. As such, we remain in an advantageous position to implement defense strategies against potential future exploitation targeting the supply chain.

Name	Initial Stage	Communication	With Supply Chain
XDSpy [65], [66]	T1566.002, T1566.003, WH	EHT	No
Modified Elephant [67]	T1566.002, T1566.003	T1587.001	No
Cozy Bear [68], [69]	T1566, WH	T1568.002, CPS	Yes
TajMahal [24], [70]	T1566	EHT, CDP, HB	Maybe
Fancy Bear [71], [72]	T1589	DF, T1573	Yes
Lazarus Group [73], [74]	T1592, T1590, T1591	T1090, T1573, T1608	Yes
Equation Group [25]	T1592, T1590, T1595	CPS, T1001	Yes
Comment Crew [75]	T1592, T1590, T1595, T1589	T1584.006, T1573, T1568.002	No
Hidden Lynx [76]	T1592, T1590, T1595, T1589, T1566	T1001.002, T1573	No
Winnti Group [77]	T1595.002	CPS, HSP, T1568.002, T1090, T1584.004	Yes
DarkHotel [78]	T1589, T1593	WET, T1566, T1584.006	No
OceanLotus [26]	T1592, T1591, T1566	HB, T1568.002	No
Turla Group [27]	T1591, T1589	T1584.006, T1090.001, SCC	Yes
Supermicro Hardware [44], [79]	T1195.003	HB	Yes

TABLE II: Undetected APT Attacks Investigation Part II

The technique names that start with T follow MITRE ATT&CK framework

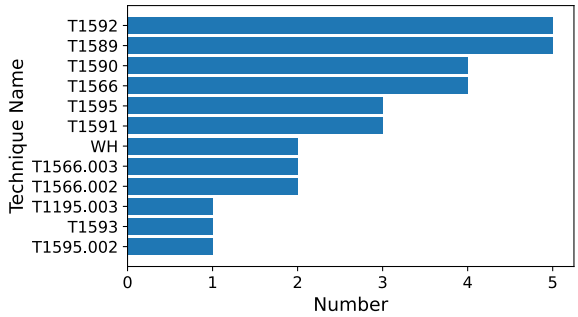

Fig. 5: Top-Technique Ranking in Initial Stage

2) Hidden Techniques: The term 'hidden technique' refers to a variety of strategies through which an adversary may conceal their actions. Such activities might encompass concealed communication with an external server, such as a C2 server, to either receive instructions or exfiltrate data. Additionally, to evade detection, malicious entities may simulate legitimate traffic or processes. Furthermore, they may delete relevant logs or artifacts to inhibit tracking. These collective techniques substantially contribute to the challenge of detecting APT activities, contributing to their longevity post-initial access. Communication concealment plays a particularly crucial role among these activities to accomplish malicious objectives. Given this assumption, we have collated communication techniques as a principal component of hidden techniques.

As illustrated in Figure 6, four techniques are employed with high frequency. T1573 and T1568.002, each with an occurrence of four, are prominent. T1573 pertains to the use of encrypted channels wherein adversaries encrypt C2 traffic utilizing specific encryption algorithms. These algorithms could be symmetric cryptography, such as AES or DES, or asymmetric cryptography, like RSA or DSA.

Conversely, T1568.002 refers to the utilization of domain generation algorithms (DGAs) under dynamic resolution. DGAs serve as a prevalent method in C2 communication, enabling adversaries to dynamically identify a destination domain for C2 traffic [81]. This technique boasts several advantages, including evasion of detection, hindrance of tracking, and prevention of takeover by defenders.

T1584.006 and HB, each appearing thrice, follow closely. T1584.006 signifies the compromised infrastructure of web services, which plays a vital role in phishing and C2 activities. For the C2, it provides traffic encryption or obfuscation and can perplex defenders by communicating with compromised legitimate web services. Additionally, it serves as a crucial component for launching watering hole attacks [32]. The HB technique, previously detailed in part III of section II, is utilized in communication scenarios for maintaining persistent access to the target's website by the adversary. Complementing the collection of hidden techniques sourced from blogs and whitepapers, we have also examined novel hidden behaviors reported in academic research.

A study inspired by the use of information hiding in APTs explored the potential application of steganography techniques in network communication protocol-based supply chain attacks on Industrial Control Systems (ICSs) [61]. The authors took the OPC Unified Architecture (OPC UA) as an example to experiment with attack vectors in Programmable Logic Controllers (PLCs). The OPC UA is a manufacturer-independent communication protocol with three different modes that can add additional signed integrity, authenticity, or encryption. By accessing the OPC UA response, it is possible to embed a hidden message into the source timestamp to inject commands. The study found that traditional Intrusion Detection Systems (IDS) are not capable of analyzing network traffic generated in hidden communication channels, which is an exemplary XOR-encrypted steganographic (stego) channel. Instead, they designed a pattern recognition-based anomaly detection approach with handcrafted features and a classifier. This approach can capture the network traffic created from the hidden channel with high accuracy and low false positives, which are highly dependent on the sequence size of packages. Potential trajectories for future research identified include devising more sophisticated methods to achieve closer digital distributions to stego messages and examining other protocols or channels via protocol converters.

A novel feature set was proposed by [82] to evaluate the efficacy of privacy protection offered by encrypted Domain Name

System (DNS) schemes. They implemented a traffic analysis attack via monitoring and censoring mechanisms. Throughout their experiments, they utilized the n-gram algorithm on traffic to generate features. This algorithm facilitates the splitting of string sequences into tokens, combining groups of n tokens into a singular feature. It aids in identifying patterns in request-response size pairs, as well as the local order of packet size sequences. The detection procedure was regarded as a supervised problem for training purposes, and the Random Forest algorithm was enlisted as the classifier to carry out classification tasks. Their findings demonstrate that encryption methods such as DNS-over-TLS (DoT) and DNS-over-HTTPS (DoH) do not offer adequate privacy protection against domain identification through traffic analysis. Furthermore, standardized padding schemes were deemed inefficient in preserving privacy. However, the study did not take into account other DNS-related attacks for privacy inference problems, nor did it offer a comprehensive analysis of the effectiveness of the DoT and DoH protocols in safeguarding against other types of attacks. Consequently, no mitigation solutions were provided for the identified attacks.

The study [83] embarked on an exploration of a wide array of DNS applications and devices to identify vulnerabilities within the DNS resolution chain. This was achieved by introducing malicious payloads, comprising special characters such as slashes, at signs, and zero-bytes, through DNS records. The authors observed that most DNS applications generally lacked input validation and parsing context checks. The DNS records examined include Pointer Record (PTR), Sender Policy Framework (SPF), TXT records, and other available DNS record types. Furthermore, it was discerned that existing validation strategies rely on validations from lower layers rather than the applications themselves. Such vulnerabilities potentially empower attacks with the ability to inject malicious payloads and fulfill their objectives via the automatic DNS resolving process. In response to these significant challenges in open DNS applications, the researchers proposed solutions that encompass the central transparency-related principles of Internet systems development. Moreover, they suggest the integration of validation layers into these applications to improve their resilience against potential breaches.

Although various APT groups may employ differing sub-techniques, a degree of commonality can be identified in techniques utilized by most APT groups. The question arises as to what these shared techniques are and how defenders can potentially leverage this information. This prompts a deeper investigation into the correlation between APT groups and the techniques they exploit to obtain a comprehensive understanding of their operational relationships.

3) Graph-Based Analysis: To derive additional insights from the assembled techniques, we conduct a graph-based analysis designed to encapsulate the complex relationships between different techniques and APTs. Our methodology applies the Python library networkx[11] and the Neo4j tool[12] to delineate the topographical relationships among the various

[11]https://networkx.org/
[12]https://neo4j.com/

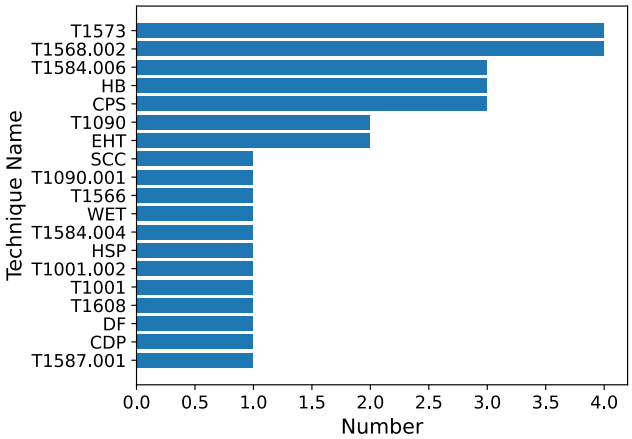

Fig. 6: Top-Technique Ranking in Communication

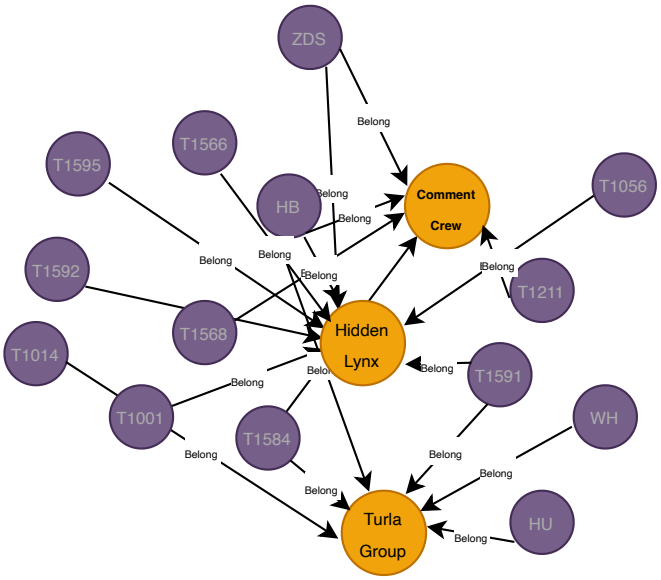

Fig. 7: Graph Visualization of APT Techniques with Neo4j - Central Part

techniques employed by APTs. Notably, networkx, renowned for creating, manipulating, and studying the structure and dynamics of intricate networks, is harnessed to conduct code-level graph analysis. Meanwhile, Neo4j, which serves as a blend of a graph database and a visualization instrument, facilitates the discovery of patterns and insights across an extensive array of data connections, enhancing both visualization performance and processing capacity for large datasets. Utilizing Neo4j, we record the visual interconnections of different APTs and identify overlapping areas of exploited techniques [84].

Based on the techniques collected in Table I, we establish a mapping between individual techniques and APT names to format edges in graphs, where the APT name serves as the source and the technique name as the destination for each edge. The relationship between them is defined as "belongs_to" from nodes to sources, corresponding to the method of named entity recognition mentioned in [84], [85]. The

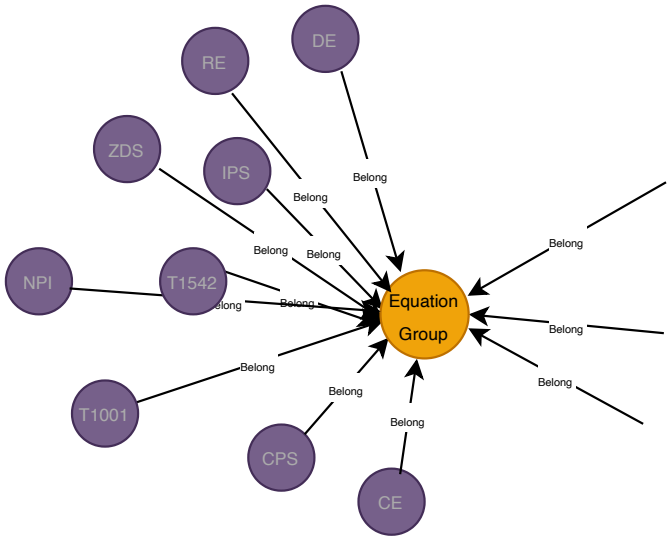

Fig. 8: Graph Visualization of APT Techniques with Neo4j - Edge Part

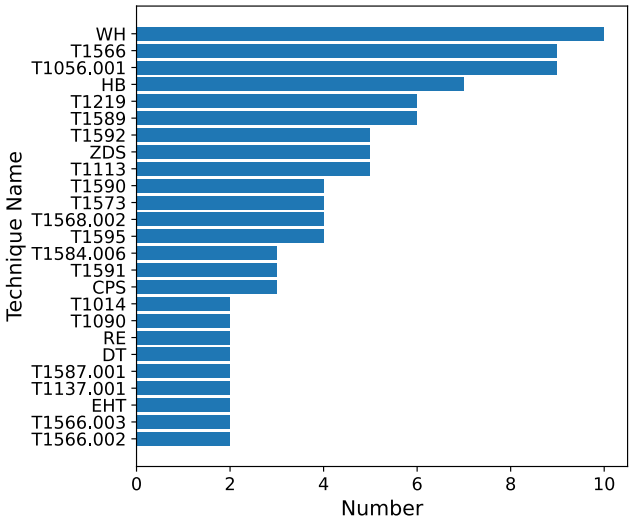

Fig. 9: Top APT Techniques Ranking

underlying principle is to identify all the techniques associated with a particular APT and ascertain the relationships between them. The ultimate objective is to deploy enhanced defense mechanisms and classify different APT groups or families for improved cybersecurity measures.

Due to space limitations, we present a partial representation of the graph in Figure 7 as the central part, and Figure 8 as one edge part, extracted from the original graph. The insights derived from this visualization can be summarized as follows:

Firstly, by observing the edge clusters, such as the Equation Group in Graph 8, we can potentially identify complex use scenarios and predict tendencies for further similar exploitations. This information empowers defenders to position themselves advantageously for additional research and defense measures. For instance, after the 2017 WannaCry ransomware attack [86], researchers found that the exploits originated from those used by the Equation Group.

Secondly, analyzing the source nodes in the central part of Graph 7, we can deduce strong correlations between APTs like Hidden Lynx, Turla Group, and Comment Crew due to their high overlaps. The specifics of these correlations lie beyond the scope of this survey; however, this information is valuable for the triage of different APT groups or families.

Finally, we observe the overlapped techniques and check the out degree of destination nodes. The out degree represents the number of edges going out of a vertex in a directed graph. This insight aids defenders in deploying prioritized solutions for overlapped malicious exploitations, considering a balanced consideration of cost and defense efficiency in most cases.

We conduct further analysis by utilizing networkx to explore the overlapped techniques in the dataset. Firstly, we construct graphs based on the edges obtained from the previous analysis. We then extract all the technique nodes with an out-degree of two and above. To visualize the distribution, we sort the techniques based on their out-degree and presented the rankings using a horizontal bar chart. Figure 9 displays the result of this analysis, showing that a particular technique labeled WH stands out with the highest out-degree. Notably, this technique differs from the outcome observed in the initial stage, indicating its widespread utilization in APT activities, although it may not always serve as the initial access technique. This may be attributed to the inherent difficulty in determining the techniques used for initial access [24], [70]. On the other hand, verifying the presence of WH in the context of an attack is comparatively easier. Academic detection research in section V provides additional insights into the significance of other techniques. Among them, T1566 and T1056.001 have garnered significant attention. T1566 pertains to the use of security software discovery, while T1056.001 refers to input capture via keylogging, wherein user keystrokes are exploited to intercept potential credential information during input. Although keylogging is typically a part of malware functionality, it can also function as an independent tool [87]. The remaining techniques are left for readers to explore. In summary, our analysis reveals that APTs share a substantial portion of techniques, accounting for 65.4% of the total collected techniques. This suggests that prioritized defense strategies have the potential to defend against the majority of APT activities.

To investigate potential differences among specific APTs based on SCVs, we conduct a graph analysis using high out-degree extraction techniques in APTs that exploit supply chain weaknesses. The technique ranking, as depicted in Figure 10, revealed that T1195.002, corresponding to software-based supply chain compromise, ranked highest. This finding aligns with the information presented in Mandiant's report [1], which also indicates a similar percentage of T1195.002 usage in supply chain-based compromises. Consequently, we can infer that software-based supply chain compromise remains the predominant exploitation method, surpassing hardware, dependencies, and development tools compromise.

The next most prominent technique is Unauthorized Access (UA), which enables adversaries to gain unauthorized access through various means, although specific details are not explic-

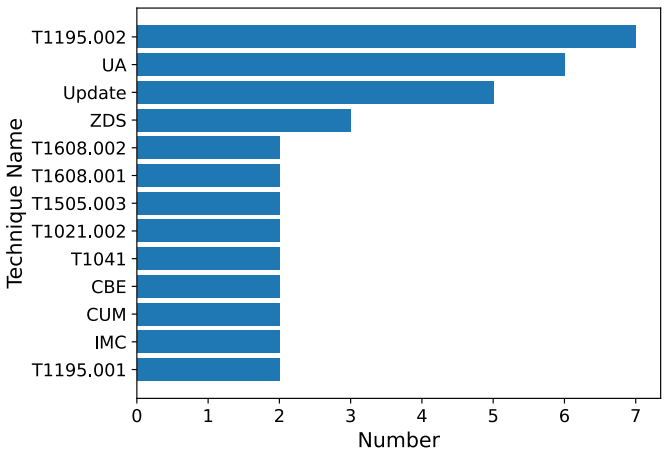

Fig. 10: Top APTs on Supply Chain Techniques Ranking

itly outlined. Nevertheless, it is evident that adversaries exploit the supply chain to expand their impact and advance their exploitation efforts. The Update Exploitation subset under UA corresponds to the broader Event Trigger Execution (T1546) technique. It illustrates how third-party vulnerabilities are introduced into a target's system or how concealed exploitation is triggered. Notably, this technique underscores the common practice of hijacking updates in supply chain-based exploitation.

Upon comparing the techniques identified in Figure 9 with those in our supply chain-specific analysis, a distinct discrepancy becomes evident. Consequently, it is imperative to develop specific defense strategies tailored to APTs that exploit SCVs.

C. Threat Model and Assumptions

1) Threat Model: Based on our previous analysis, it can be inferred that software supply chain (SSC) remains the primary method of exploitation in the supply chain context. To ensure the comprehensiveness of the threat model, we specifically focus on SSC [13], [16] by combining general APT attack stages. Additionally, our emphasis is on open-source software, representing a more prevalent scenario in SSC [5]. Furthermore, we incorporate the software engineering lifecycle with the package manager to provide a comprehensive model [14].

The decision to exclude hardware supply chain (HSC) is based on the significant diversity of devices and infrastructures involved [22], [46], [60], making it challenging to create a single threat model that covers all HSC cases. Focusing on open-source SSC enables us to understand the goals and potential attack paths in constructing the threat model. As described in Part I of Section II, APT is defined as a targeted attack. However, the majority of supply chain attacks are non-targeted, with the main goals of widespread impact or compromising integrity [16]. In Table I in Part II of Section IV, we identify the goals of some long-term undetected APT attacks, with espionage being the predominant aim in most investigated APT activities. By integrating the techniques demonstrated in Table II and the statistical analysis in Figure 10, we present the threat model in Figure 11.

Within the threat model, the injection of malicious code serves as the initial exploitation method [16], aiming to gain unauthorized access while maintaining concealment. The introduction of this injected code to the target system depends on user actions. Various scenarios, such as library dependencies, updates, and downloads, exist [5], [13], [52], [54], [62]. For example, users may inadvertently introduce infected versions of Python libraries as dependencies, and unverified updates might automatically install malicious code. Downloads from unofficial forks can also introduce malicious commits and unpatched vulnerabilities from original repositories.

After the malicious code is inserted into the user's system, trigger mechanisms come into play to initiate its activation. These triggers can start automatically upon completion of downloads or updates. More sophisticated exploitation involves setting up specific events based on scheduling tasks, rare occurrences, or predetermined periods of concealment [4], [22]. Subsequent exploitation follows the general stages of APT activities. Triggered behaviors may involve downloading second-stage malware with enhanced functionalities [88] or establishing contact with additional services like C2 servers. Through further exploitation, the adversary may engage in reconnaissance once again to gather more internal information or conduct internal phishing. The ultimate goal is typically to acquire more credential information.

After gaining valid accounts, the malicious actor can expand their reach through lateral movement to infect more devices or machines. By obtaining administrative privilege through an admin account, the final objective is achieved by exfiltrating data to their C2 server. To avoid detection during exfiltration, adversaries may send data out in batches over multiple days and erase previous exploitation traces. Consequently, the ultimate intention of supply chain-based APT attacks is often not to compromise integrity or cause immediate damage. The widespread adoption of supply chain attacks increases the likelihood of gaining unauthorized access to targeted systems.

In this section, we present a typical threat model focusing on common usage scenarios. We deliberately exclude specific scenarios, like exploiting watering holes to compromise a single host and spreading through the supply chain, as they do not represent the overall supply chain process. Notably, our proposed threat model lacks general reconnaissance in the first stage. Additionally, comparing techniques used in general APTs with those in APTs based on the supply chain reveals significant divergence. To achieve a more unified understanding of typical APTs, we discuss a custom kill chain targeting APTs in the supply chain.

2) Custom Kill Chain: The trade-off between detection accuracy and computational resource constraints in practical scenarios necessitates an optimal defense mechanism solution [21]. While there exists a holistic chain to describe APT stages, it's crucial to assess the data obtainable from each stage based on the characteristics of APTs in the supply chain context. In our research, we further investigate APTs within the supply chain, employing techniques to potentially segment the stages. Table III depicts some missing values due to information

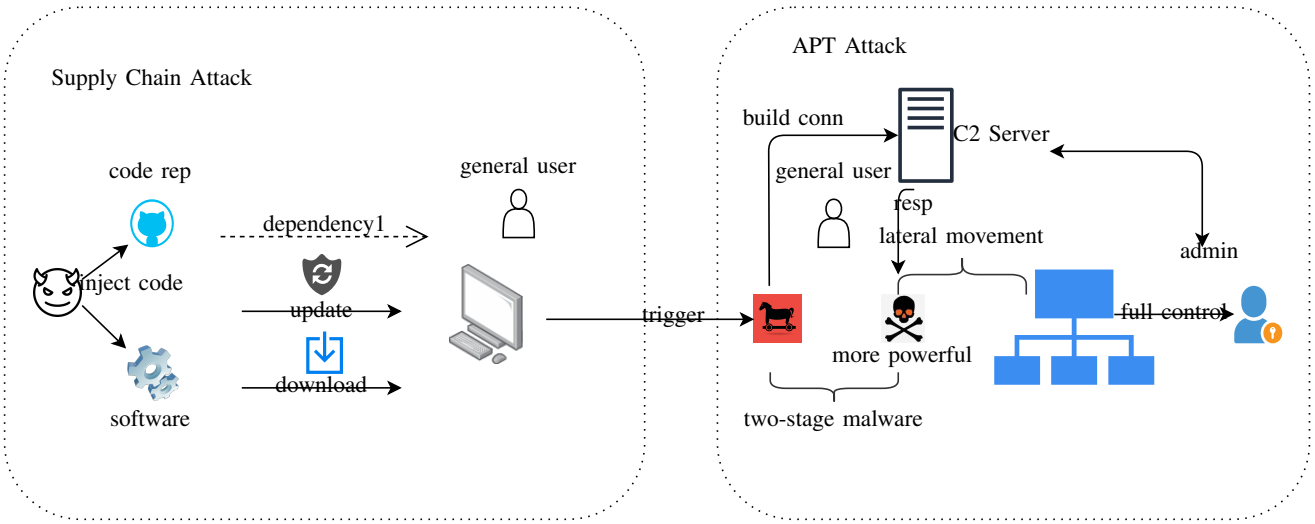

Fig. 11: Threat Model of Supply Chain based APT

Name	Foothold Establishment	Delivery	Installation	Lateral Movement	Exfiltration	Erase Evidence
SolarWinds [56], [57]	T1195.001, T1195.002, IMC, CUM, CBE	Update	T1021, Activate Backdoor	T1550, T1021, T1071.001, T1078	T1041	DL, MST, OA
CCleaner [89], [90]	T1195.002, CUM, UA, IMC, CBE	Update	Download and Execute Software	N/A	T1041	N/A
ASUS [91], [92]	UA, T1195.002	Software Package, Update	Initial Setup or Update	N/A	N/A	N/A
NotPetya [93]	HU, T1195.002	Update	T1211	T1021.002, T1570	T1132	T1485
Kaseya VSA [94], [95]	T1195.002, ZDS, UA	T1195.002	Automatic	Update	T1574.002	N/A
Accellion FTA [96]	T1195.002, ZDS, UA	T1505.003	T1608.001,T1608.002	N/A	T1048	Not Clear
Codecov [97], [98]	T1195.001, UA	Bash Uploader	Backdoor	N/A	N/A	Not Clear
HAFNIUM [45], [99]	T1195.002, T1190, ZDS, UA	T1505.003	T1608.001, T1608.002, T1059.003	T1003.001, T1021.002	T1567.002	Not Clear

TABLE III: Supply Chain Based APT Attacks

The technique names that start with T follow MITRE ATT&CK framework

gathering limitations. We rename some technique names from blogs for realism. To ascertain genuine distinctions, we perform a series of processing and comparisons. Our primary tools include pyattck [100], a Python package for interacting with the MITRE ATT&CK Framework, to determine tactics for each technical name. Additionally, we use rapidfuzz [101], a fast string matching library for Python and C++, based on fuzzywuzzy [13], for fuzzy matching the technique names in the MITRE framework with those extracted from blogs.

For pre-processing, we took the following steps:

- Mapped abbreviations to their original names to align with MITRE techniques.
- Integrated techniques from both Table I and Table II into a single dataset, denoted as dataset1.
- Separately integrated techniques from Table III into another dataset, referred to as dataset2.
- Compiled a comprehensive list of tactics defined in the MITRE and APT stages as outlined in Section II.
- Pre-defined mapping relations between tactics and APT stages using the dictionary outlined in Listing 1.
- Removed duplicate technical names in both datasets to ensure data consistency and accuracy.

```
map_tactic_apt_dict = {
    "Reconnaissance":["reconnaissance",
                      "resource-development"],
    "Delivery":["initial-access"],
    "Installation":["execution", "persistence"],
    "Lateral-Movement":["privilege-escalation",
                        "defense-evasion",
                        "credential-access",
                        "discovery","lateral-movement"],
    "Exfiltration":["collection","exfiltration",
                        "command-and-control"],
    "Erase-Evidence":["impact"]
}
```

Listing 1: Dictionary of Mapping from Tactics to APT Stages

To compare and contrast the two datasets and identify differences and commonalities, we utilize a stacked vertical bar chart to visualize the relationship between tactics and stages as shown in Chart 12. To create the charts, we calculate the union of techniques, representing the total number of unique techniques present in both datasets without repetitions. Additionally, we determine the intersection of techniques, indicating the common and overlapping techniques between the two datasets. We extract them by creating two dictionaries wherein the keys are separately tactic names and stage names. The values are list type of techniques associated with those keys. The association between techniques and tactics or stages is based on the definitions provided in pyattck, aligning with the definitions used in the MITRE framework.

In Chart 12, we observe that the majority of techniques are concentrated in specific stages, such as persistence, privilege escalation, and defense evasion tactics. Each of these three stages has approximately 20 techniques, while initial access, execution, and command and control (C2) have a range of 15 to 20 techniques. This discrepancy suggests that there is more enriched or available trace data in these tactics,

[13] https://github.com/seatgeek/fuzzywuzzy

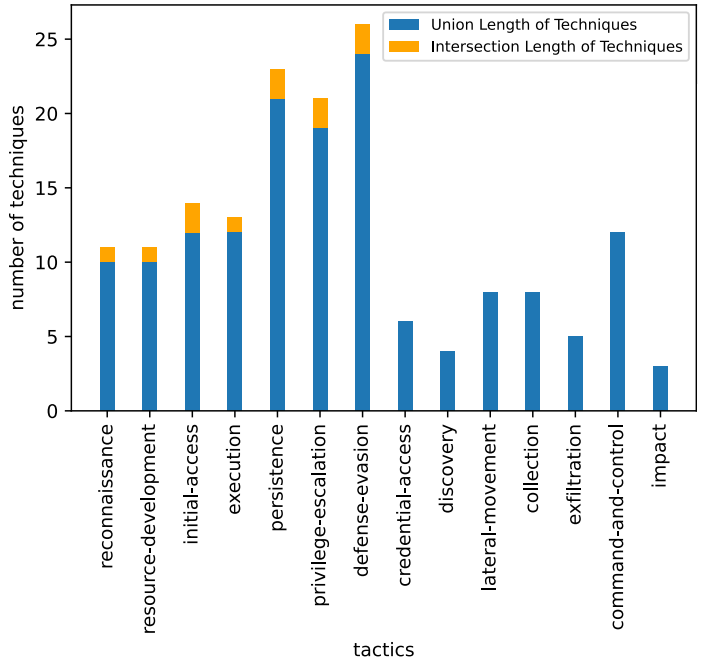

Fig. 12: Comparison of Union and Intersection Techniques in Tactics between all APTs

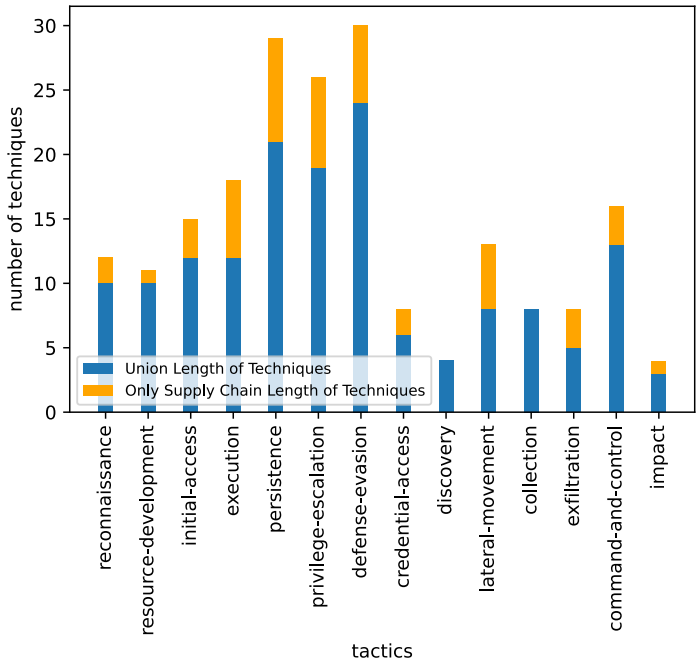

Fig. 13: Comparison of Union Techniques in all APTs and Techniques only in APT based on Supply Chain

providing defenders with a higher possibility of identifying APT activities. Analyzing the intersection part of the chart reveals that only a portion of tactics overlap in the two datasets.

To gain further insight into the distribution of these overlaps within the total pool of techniques in APTs based on the supply chain, we conduct the similar process as creating the Chart 12. Examining this chart in comparison with Figure 13, we find that the previously non-overlapping tactics after defense evasion in Figure 13 have techniques that are unique to the supply chain. This observation confirms the distinctiveness of these tactics in the context of supply chain-based exploitation. Additionally, while there are some overlaps in other stages between the two datasets, the techniques that exist exclusively in the supply chain still exhibit relatively large numbers, with the exception of the initial reconnaissance tactics. In conclusion, the technique stacks utilized by general APTs and APTs based on the supply chain show significant dissimilarities.

Based on the data in Figure 13, we categorize tactics into custom APT stages using the rule defined in Listing 1. Figure 14 illustrates that APT techniques in the supply chain domain are primarily concentrated in the installation and lateral movement stages. Conversely, the erase evidence stage exhibits the fewest number of techniques. The remaining three stages show a relatively balanced distribution of associated techniques. This observation suggests that stages with fewer techniques may introduce a higher ratio of bias or disparity. To address potential deviation in fuzzy matching, we conduct a more detailed analysis of stages with shorter bar lengths. The list of techniques grouped by stages is provided in Listing 2.

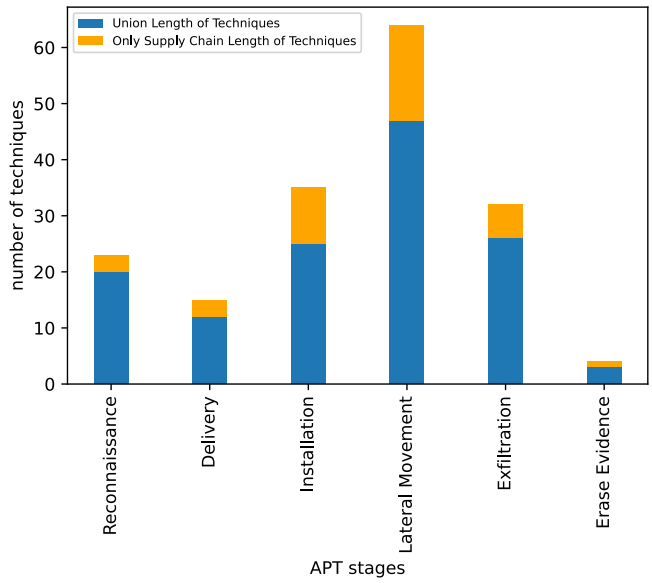

Fig. 14: Comparison of Stages between General APT and APT based on Supply Chain

```
Reconnaissance :
['Download and Execute Software', 'T1608.002',
```

```
    'Software Package']
Delivery :
['T1195.001', 'CUM', 'T1195.002']
Exfiltration :
['T1048', 'Backdoor', 'T1071.001', 'T1567.002',
    'T1041']
Erase Evidence :
['T1485']
```

Listing 2: Techniques only in Supply Chain based on Stages

Upon comparing the findings to Table III, we observe that the 'Download and Execute Software' technique is placed in the installation stage, while Table III assigns it to the reconnaissance stage due to fuzzy matching. This discrepancy arises from the high occurrence of related terms like 'download' and 'software' in the resource development tactic, leading to its misclassification. Similarly, the 'Software Package' technique is also mistakenly allocated to the reconnaissance stage, whereas it should rightly be placed in the delivery stage according to Table III.

Another notable technique is T1608.002, which involves the capabilities of uploading tools. While MITRE categorizes this technique under the resource development tactic, we propose placing it in the installation stage. Adversaries often use this technique to upload malware or tools, especially in multiple-stage attacks. This allows them to collect more information for lateral movement or execute malicious code. Powerful malware can possess multiple functions, potentially causing extensive damage [70], [96].

Additionally, we investigate technique T1485 in the erase evidence stage, involving data destruction, including the removal of logs or malware for anti-forensics purposes—common operations in various APTs. Detection in this stage poses challenges due to limited clues. Consequently, we propose a custom kill chain for detection purposes, comprising the stages of delivery, installation, lateral movement, and exfiltration. This aligns with the three constant steps proposed for APT detection in [80], offering a substantial number of clues for detection.

In the next section, we have compiled a complex tree listing all potential attack vectors based on a comprehensive investigation. This compilation incorporates insights from both academic research and industrial practices.

3) Attack Vector: The attack vectors for APTs exploiting SCVs are derived from comprehensive threat models involving various components and potential threats. Through an extensive analysis of numerous academic papers, we have developed a comprehensive set of attack vectors covering both hardware and software aspects. In the diagram presented in Figure 15, we have categorized the attack vectors into hardware and software domains, considering the varying levels of privacy and confidentiality requirements in each area. Our investigation, detailed in Table IV, reveals that the majority of research in this domain focuses on software-related attacks, with only a few works addressing hardware-related threats.

In the hardware supply chain, which is not typically public-facing, Eggers et al. [22] listed the full development cycle of high-level hardware and software, identifying six potential threats associated with various stakeholders, including end users, integrators, solution providers, and manufacturers. Additionally, Basu et al. [60] proposed a tailored threat model for the electronics supply chain, describing potential usage of malicious tools in different development steps, such as high-level synthesis, logic synthesis, physical design, and verification.

Reference	What Type	Detect What Attacks
[5]	software	trojan
[12]	software	malware attack
[59]	software	all potential attacks
[13]	software	injected code
[22]	hardware	all potential attacks
[14]	software	malware
[15]	software	vulnerabilities
[46]	hardware	injection of malicious code

TABLE IV: Classification of Academic Research Objects and Attacks in Supply Chain Attacks

On the software side, Jiang et al. [59] investigated the software cycle involving pre-trained models stored in model hubs like Hugging Face. They identified potential stakeholders, such as hub maintainers and registry maintainers, and explored the relationships between threats and components linked to these stakeholders. In a separate study, Yang et al. [102] delved into stealthy attacks targeting code models. Their focus was on adaptive backdoors with adversarial features to inject triggers into inputs, presenting a novel attack vector for which state-of-the-art defense methods may not be effective. Additionally, Duan et al. [14] explored supply chain attack scenarios in package managers for interpreted languages, presenting potential attacks at individual stages of the package development cycle.

Additionally, we include cases of closed-source software from Barr-Smith et al. [5]. Their research investigated supply chain attacks and demonstrated potential vectors through static and dynamic analyses. The study covered threats targeting the file system and explored living-off-the-land (LotL) techniques, which are evasive mechanisms used in trojanized binaries.

Compared to the exhaustive attack vector proposed in [16], our model introduces a stakeholder dimension and reveals connections among stakeholders, components/processes, and potential threat attacks. Our model includes 12 unique stakeholders, 38 components/processes, and 42 threats. Although we have indexed fewer threat vectors compared to the 107 in [13], the potential connections among different stakeholders, components, and threats in our model are significantly more extensive. Additionally, our survey is the first to provide a relatively comprehensive enumeration of hardware attack vectors, considering their unique attributes in different scenarios. It's worth noting that in some scenarios, the integration of hardware and software makes it challenging to completely distinguish between them [22].

The enumeration of attack vectors serves various practical applications, including threat modeling, awareness training, and risk assessment [16]. Additionally, it plays a crucial role in scoping penetration tests, contributing significantly to enhancing an organization's security landscape.

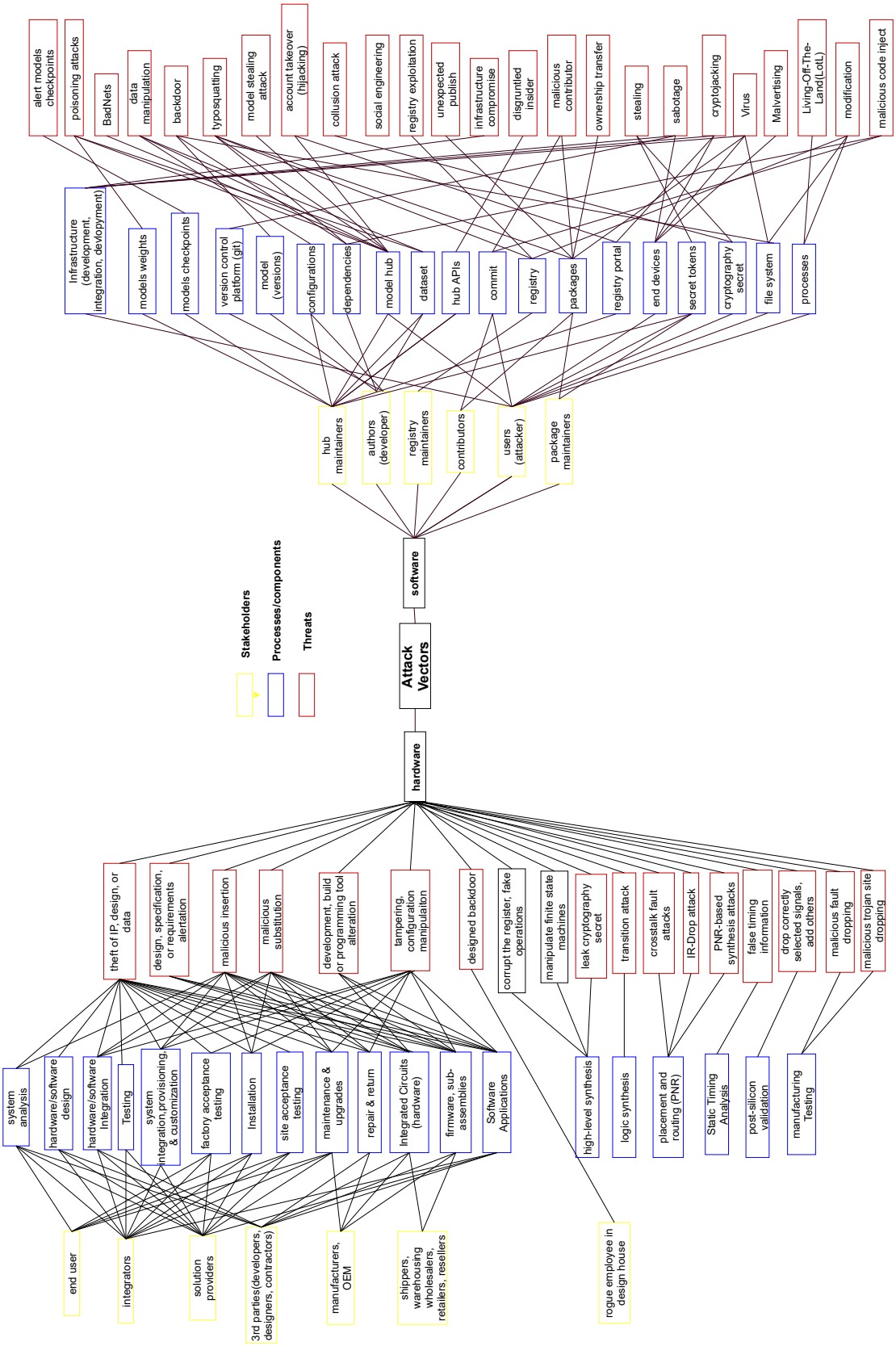

Fig. 15: Supply Chain Attack Vector

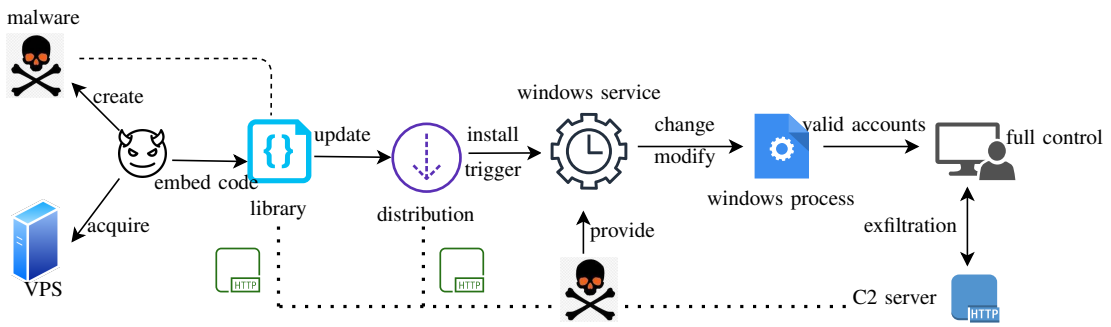

Fig. 16: SolarWinds Attack Scenario

D. Case Study

In this section, we present two case studies—one focusing on the hardware supply chain (HSC) and the other on the software supply chain (SSC)—to illustrate nearly real-world examples of exploitations in supply chains. These case studies shed light on the vulnerabilities and potential threats within both types of supply chains, providing valuable insights into the challenges faced by organizations in securing their supply chain processes. By examining these specific instances of attacks and their impact, our goal is to enhance our understanding of the complexities involved in mitigating supply chain risks. This emphasizes the significance of implementing robust security measures to safeguard against potential breaches and disruptions.

1) SolarWinds: The SolarWinds supply chain attack stands out as a prominent example of exploitation within the software supply chain (SSC) and has been extensively studied in both academia and the cybersecurity industry [4], [56], [57]. Numerous research efforts have contributed to understanding the attack's intricacies and devising effective countermeasures.

The work by Alkhadra et al. [103] offers a general analysis of the infection process and the affected sections but does not delve into detailed technical aspects. In contrast, Martìnez et al. [104] present an in-depth case study, incorporating the cyber kill chain and providing a comprehensive attack timeline. Their research includes a literature review to bolster result credibility and suggests countermeasures to prevent similar attacks, some of which align with the SLSA framework. Peisert et al. [105] contribute valuable insights from an academic perspective, emphasizing the critical role of integrated software or hardware in the success of SSC compromises. They propose solutions such as deeper analysis techniques and code provenance tracking to mitigate such risks. The case study conducted by Barr-Smith et al. [5] focuses on the SolarWinds attack's trojans, Sunburst and Supernova. Their analysis, based on the attack graph in Figure 16, seeks potential attack clues and traces to aid detection, particularly for advanced hidden techniques employed.

The diagram in Figure 16 showcases the Sunburst malware, a trojanized version of the SolarWinds Orion plug-in. This malware includes a backdoor and communicates with third-party servers via HTTP. Developed during the resource development tactic, the malware employs a virtual private server (VPS) to provide multiple IP addresses, aiming to evade detection. The choice of IP addresses from the victim's country further complicates detection. However, anomalies in the rate of travel for IP access or logins from unusual ASN may signal potential malicious activity.

The malware operates by retrieving and executing commands after a dormant period of up to two weeks. To disguise network traffic as legitimate protocol, communication with the extra server is employed. Reconnaissance information is stored within the plugin configuration file. Lateral movement is facilitated using valid credentials, and graphed logon activities may reveal one-to-many relationships indicative of malicious access.

To evade detection during payload execution, the adversary employs temporary file replacement and temporary task modification techniques. Detecting such behaviors involves examining logs for SMB sessions, where a delete-create-execute-delete-create pattern may be observed in a short time. In-depth malware analysis, detection of Domain Generation Algorithm (DGA) usage, and disguised network traffic detection are also potential methods to uncover exploitation. While these detection methods may introduce bias, proper correlation can help reduce false positives.

2) Nuclear Power: The nuclear power case study is derived from information on the well-known hardware-based APT attack known as Stuxnet [14]. This case is employed to illustrate a potential attack scenario in the hardware supply chain (HSC). Stuxnet was a targeted threat specifically aimed at industrial control systems (ICS) such as gas pipelines and power plants. Its objective was to reprogram ICS by modifying code on programmable logic controllers (PLCs). According to Symantec's report [106], Stuxnet utilized multiple zero-day exploits, a Windows rootkit, anti-virus evasion techniques, and command and control (C2) mechanisms. The most likely initial access point was through infecting a willing or unknown third party, making it relevant for our HSC scenario. The infection technique primarily involved DLL injection. Additionally, we refer to existing research on hardware supply chain attacks in nuclear power infrastructures [22], [107] to develop a more explicit attack scenario.

In the provided diagram (Figure 17), we present the entire life cycle of hardware development, including integration into end-user systems. The connection between the PLC and the final product centrifuge can be established during the hardware

[14] https://attack.mitre.org/software/S0603/

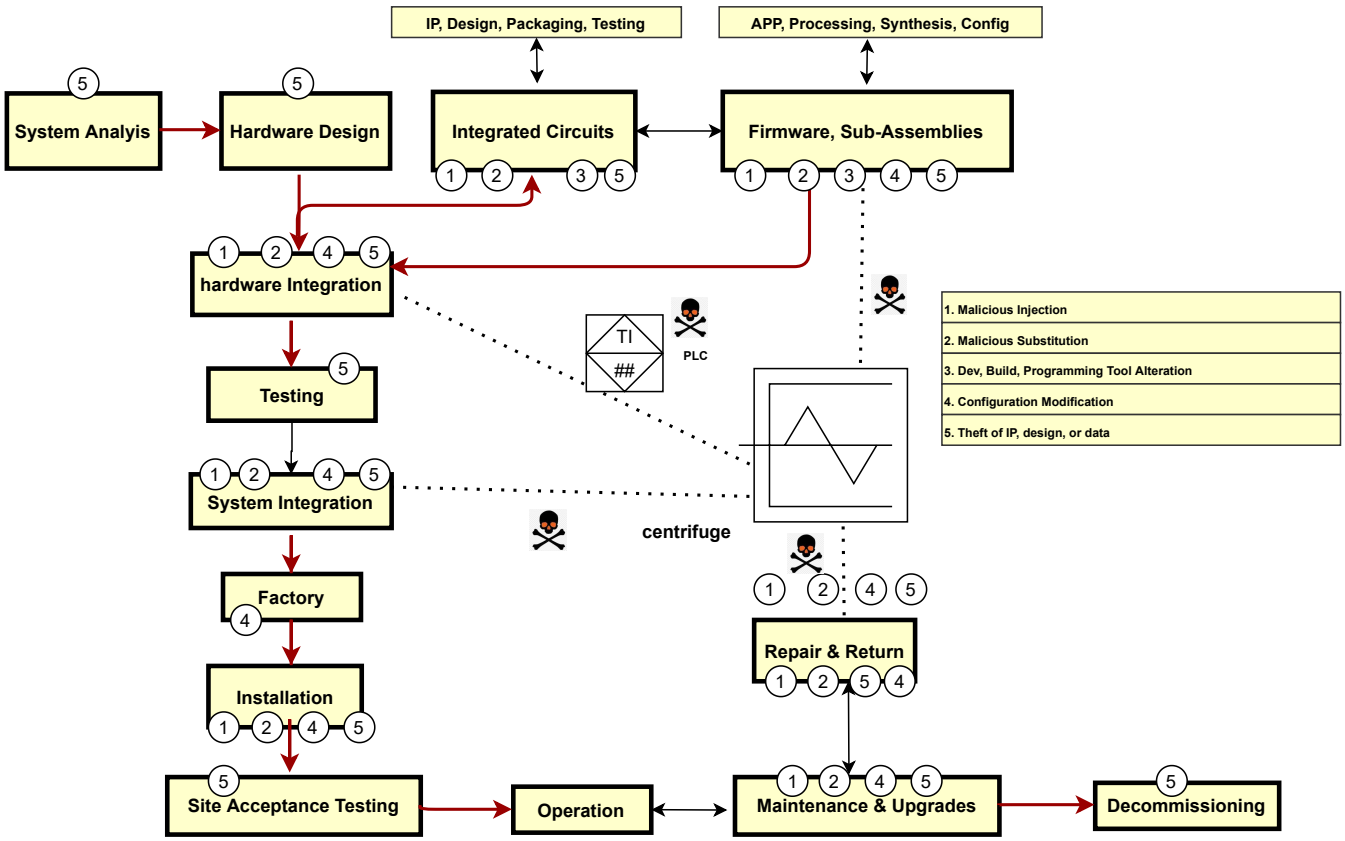

Fig. 17: Nuclear Power Attack Scenario

integration process. However, malicious code can also be introduced at various stages, such as firmware, sub-assemblies, and system integration steps. Different stages of development may be targeted by diverse attacks. We have identified five critical threats, including malicious injection (1) or substitution (2), tool alteration (3), configuration modification (4), and theft of IP or data (5) [22], [106]. These attacks can be launched at any stage, whether it is during specific building or testing phases, or even during the delivery process. The reconnaissance stage in HSC primarily involves the theft of schematics of ICS, where each PLC is uniquely configured. Hence, successful attacks involving data or design theft can contribute to reconnaissance in any stage. The development tool with inserted malicious code can introduce a payload during compiling or integration processes, aligning with build threats outlined in the SLSA framework. By enumerating all the processes, it becomes clearer for malicious actors to identify which threats can be executed at different stages. Additionally, tailored defense policies can be deployed accordingly. Symantec [15] previously set up a system to monitor traffic to Stuxnet C2 servers to observe infection rates and identify infected computer locations. However, in the case of ICS, due to the strong privacy and confidentiality requirements, direct traffic monitoring may not be feasible. Therefore, we recommend implementing code-based checking before integration and adopting the compromised DevSecOps approach [108] during the building process for enhanced detection and prevention.

[15]https://www.broadcom.com/products/cybersecurity

V. Classification of Supply Chain Based APT Defense Methods

To illustrate the latest trends in defense against APTs based on SCVs, we conduct an extensive review of scholarly articles from top-tier journals and conferences in the fields of APT detection and supply chain-based attacks published over the past three years. The comprehensive statistical analysis is presented in Table V, covering both domains. The first half of the references up to Exorcist [5] highlights the most recent defense methods in APT detection. The remaining references stress on the defense methods against supply chain attacks. We record the detection methods employed and whether the introduced methods are historical data or pattern-based. This dimension indicates whether a large amount of training data is required for the implementation of detection. By integrating all the analyzed papers, we have formulated a triage tree in Diagram 18 that includes all investigated detection methods in recent years. This triage tree serves as a useful guide for the development and deployment of defense products against APTs based on SCVs, providing valuable insights and instructions to researchers and practitioners in the field.

A. Detection Methods

This category encompasses research papers primarily focusing on APT detection. Within this category, we have further di-

Defense Methods	causality	provenance	cross-host correlation	novel features	deep learning	rule-based	differential analysis	graph inference	anomaly scoring	state-based track	semantic analysis	machine learning	risk management	program analysis	attack vector enumeration	block chain	history/pattern based
SteinerLog (2022) [8]	✓	×	✓	×	×	×	×	×	×	×	×	×	×	×	×	×	✓
TRACE (2021) [9]	×	✓	✓	✓	×	×	×	×	×	×	×	×	×	×	×	×	×
Hawk-Eye (2021) [6]	×	×	×	✓	×	×	×	×	×	×	×	×	×	×	×	×	✓
ATLAS (2021) [7]	×	✓	×	×	✓	×	×	×	×	×	×	×	×	×	×	×	✓
RapSheet (2020) [10]	×	✓	×	×	×	×	×	×	×	×	×	×	×	×	×	×	✓
Mnemosyne (2020) [32]	✓	×	×	×	×	×	✓	×	×	×	×	×	×	×	×	×	✓
Hopper (2021) [109]	×	×	×	×	×	✓	×	✓	✓	×	×	×	×	×	×	×	✓
KCSM (2021) [17]	✓	×	×	×	×	✓	×	×	×	×	×	×	×	×	×	×	×
CONAN (2022) [80]	×	✓	×	×	×	×	×	×	×	✓	×	×	×	×	×	×	×
Euler(2203) [110]	×	×	×	×	✓	×	×	×	×	×	×	×	×	×	×	×	✓
Niu W. (2021) [111]	×	×	×	×	×	✓	×	✓	×	×	×	×	×	×	×	×	×
Yan G. (2020) [112]	×	×	×	✓	×	×	×	×	×	×	×	×	×	×	×	×	✓
Poirot (2022) [113]	×	×	×	✓	×	×	×	×	×	×	×	×	×	×	×	×	×
Exorcist (2022) [5]	×	×	×	×	×	×	✓	×	×	×	×	×	×	×	×	×	–
yeboah (2019) [12]	×	×	×	×	×	×	×	×	×	×	×	✓	×	×	×	×	–
Jiang W. (2022) [59]	×	×	×	×	×	×	×	×	×	×	×	×	✓	×	×	×	–
Ladisa (2022) [13]	×	×	×	×	×	×	×	×	×	×	×	×	×	✓	×	×	–
Eggers (2021) [22]	×	×	×	×	×	×	×	×	×	×	×	×	×	×	✓	×	–
MALOSS (2020) [14]	×	×	×	×	×	×	×	×	×	×	×	×	×	✓	×	×	–
HARP (2021) [15]	×	×	×	×	×	×	×	×	×	×	×	✓	×	×	×	×	–
SGX (2020) [46]	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	✓	–

TABLE V: Collection of Academic Defense Methods

✓: used ×: not used –: unclear

vided the papers into two subcategories: Attack Reconstruction and Anomaly Detection. To provide a more comprehensive understanding, we have also included insights from industrial experts in industrial solutions, which are presented in Table VIII.

1) Attack Reconstruction: Attack reconstruction involves the application of technologies to recover attack paths or graphs by analyzing the original data sources or correlating information from alerts generated by intrusion detection systems. This process encompasses three main related technologies: provenance, causality, and correlation analysis. Through the utilization of these techniques, attack reconstruction has proven to be an effective solution for mitigating potential false predictions generated in anomaly detection [8], [109].

a) Data Provenance: Data provenance refers to the documentation of the origin, creation, and propagation process of data, encompassing its lineage and derivation [114]. Due to its ability to track the history and origin of information throughout its life cycle, data provenance has found significant use in the field of cybersecurity, particularly in distributed environments [115]. In the context of APT detection, data provenance can offer valuable insights into specific scenarios.

To address efficiency concerns associated with provenance analysis, [80] proposed a state-based framework called CONAN, which does not store historical data. This approach eliminates unnecessary phases of APT attacks and only retains approximately one in a thousand events in a database. As a result, it efficiently detects both known and unknown APT attacks with minimal false positives. However, debates exist regarding the framework's ability to detect APT attacks employing new or unknown techniques, its representation of all real-world scenarios, and the required computational resources for real-time processing of large data volumes.

In [10], the authors integrated data provenance into existing industrial Endpoint Detection and Response (EDR) systems to combat alert fatigue. They introduced Tactical Provenance Graphs (TPGs) to identify causal dependencies of security alerts generated by the EDR system. Furthermore, they developed a new threat scoring methodology to improve accuracy and maintained a minimally-sufficient skeleton graph to reduce storage overhead. The system overcame limitations of existing systems with cross-machine analysis, adaptive attacks, and simulated APT exercises. However, its effectiveness depends on the comprehensive and accurate alerts generated by the EDR. Future work aims to incorporate online analysis with incremental updates to TPGs and expand data sources to enhance performance.

A novel attack investigation framework, ATLAS, was proposed in [7]. This system addresses the issue of alert fatigue resulting from a large number of threat alerts. It integrates natural language processing (NLP) and machine learning (ML) techniques for data provenance analysis to distinguish attack and non-attack sequences. By employing NLP techniques such as word embedding and lemmatization, contextual event sequences are mapped to numerical vectors. The Long-Short Term Memory (LSTM) model is then trained to identify

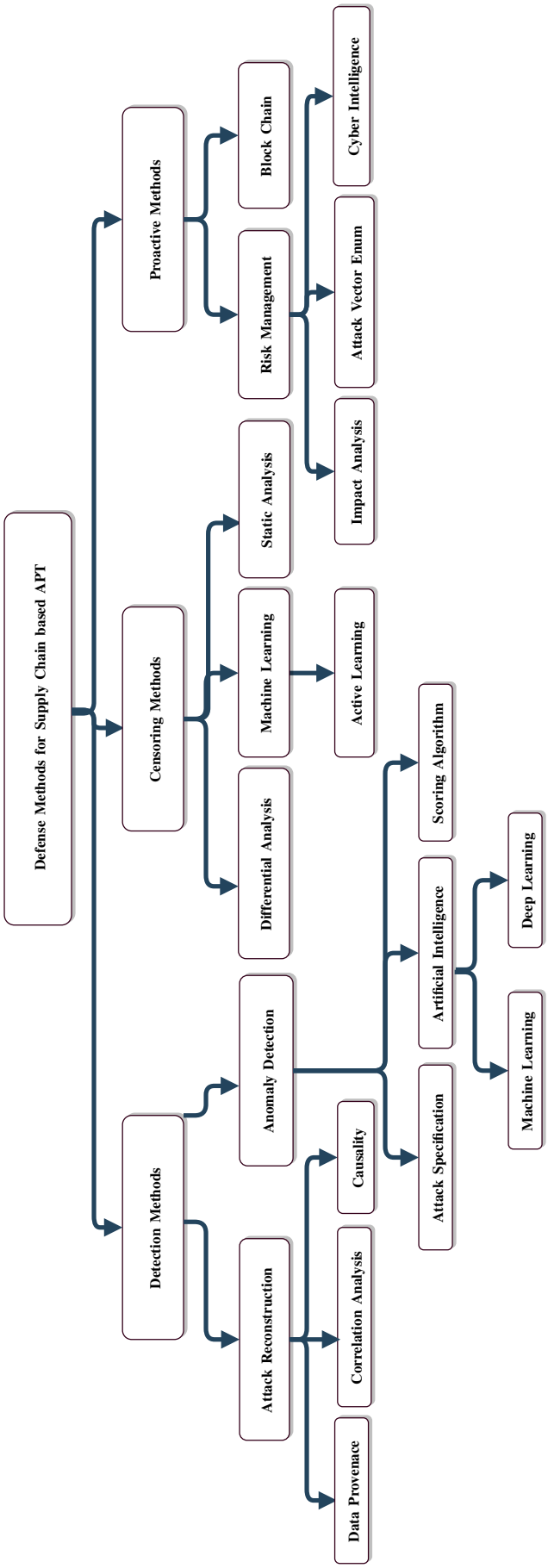

Fig. 18: Classification of Defense Methods

backward and forward relationships between events within sequences. The data provenance in this system helps reconstruct the attack story by establishing causal graphs based on identified attack events. The approach outperforms intrusion detection and association analysis methods by precisely locating critical steps and learning historical attack patterns. Nonetheless, this research is limited by reliance on single-source data (audit logs), the detection of attacks that hide their behavior within sequences of normal events, human efforts required to initiate investigations, and model correctness dependent on training log entries.

The TRACE provenance tracking system, as discussed in [9], seems to be a promising approach for efficient APT stage detection at an enterprise level. The integration of static analysis, continuous improvement efforts over several years, and the goal to construct an enterprise-wide causal graph for APT activity identification are noteworthy. It's particularly interesting that the system has undergone practical testing in adversarial engagements, achieving an 80% detection coverage for APT stages. The potential future enhancements, such as integrating an in-kernel cache-based audit logging system and model-based causality inference, indicate a commitment to ongoing improvement.

Data provenance, while offering valuable insights into the origin and propagation of data, does indeed face challenges, as mentioned in [80]. The efficiency, memory, computation, and storage resource challenges become more pronounced, especially in real-world, long-term APT scenarios. Balancing the need for comprehensive analysis with the constraints of resource usage is a critical consideration for the practical implementation of data provenance in cybersecurity defense strategies. This highlights the importance of continuous research and development efforts to address these challenges and optimize the performance of data provenance analysis in the context of APT detection.

b) Causality: Causality, as it relates to understanding cause-and-effect relationships between events, involves both causal discovery and causal inference. Causal discovery aims to infer causal relationships among variables from observational data when no existing relationships are assumed [116]. On the other hand, causal inference quantifies the influence of one variable on others, relying on prior assumptions about their relationships [117]. These concepts, rooted in probabilistic theory and statistics, play a crucial role in understanding and modeling causal relationships in various domains, including cybersecurity where they can contribute to effective APT detection and response strategies.

Causal discovery methods are diverse, encompassing various approaches to infer causal relationships among variables. Here are some categories and examples:

- Constrained-Based Methods:
  - Principles: These methods use principles of independence and conditional independence to assess statistical relations between variables.
  - Examples: PC (Peter and Clark), FCI (Fast Causal Inference), and CD-NOD (Causal Discovery with Non-Gaussian Overlapping Data) are examples [118]. CD-NOD is particularly designed to handle heterogeneous data [119], [120].
- Score-Based Causal Discovery:
  - Principles: Involves scoring candidate causal graphs and selecting the one with the highest score.
  - Example: Greedy Equivalence Search (GES) is a typical implementation in this category [119].
- Constrained Functional Causality Model:
  - Principles: Explores cause-effect relations assuming functional relationships.
  - Examples: Post-Nonlinear Causal Model, Linear Non-Gaussian Acyclic Model [121], and Additive Noise Models are instances in this category [122], [123].
- Implicit Causal Representation Learning:
  - Examples: Generalized Independent Noise (GIN) model and Hierarchical Latent Representation Learning fall into this category [124], [125].

These methods provide different ways to approach causal discovery, each with its strengths and considerations depending on the characteristics of the data and the context of the analysis.

In the context of causal inference, graphical causal models play a crucial role in gaining clear insights into observational data and understanding interrelations [126], [127]. These models are especially valuable for representing causal relationships among variables. In APT detection, causal analysis has been applied to construct causal graphs, facilitating relationship and influencing factor analysis within the observed data. Causal graphs are graphical representations that depict the causal relationships between variables. These graphs help in understanding the cause-and-effect connections among different factors, making them a powerful tool for causal inference. By constructing causal graphs in APT detection, researchers and analysts can uncover patterns, dependencies, and potential causal links between various events or variables related to advanced persistent threats. The application of causal analysis in APT detection allows for a more nuanced understanding of the dynamics at play, potentially leading to improved detection methodologies and more effective countermeasures against sophisticated cyber threats.

The study presented in [8] focuses on developing an automatic solution for detecting and reconstructing an attack campaign, particularly those associated with APTs. The SteinerLog system is introduced as a tool to achieve real-time detection of APT campaigns by correlating cross-host attack activities using a hierarchical graph traversal approach. They developed the SteinerLog system, which employs the prize collecting Steiner tree algorithm to detect APT campaigns in real-time by correlating cross-host attack activities with hierarchical graph traversal. Risk scores are assigned based on rarity and sparsity scores of anomaly events. Temporal traversal is used to build the attack stages, and historical attack patterns and customer behaviors are incorporated to enhance detection accuracy and reduce investigation time. Despite these advancements, the study highlights several challenges faced by the SteinerLog system, including evasion attempts by sophis-

ticated threat actors, the absence of Tactics, Techniques, and Procedures (TTPs) tags in anomaly detectors, and limitations in detecting stealthy long-term activities. These challenges underscore the evolving nature of APTs and the need for adaptive and robust detection mechanisms to stay ahead of cyber adversaries.

In [7], causal graphs were constructed from recognized attack events in audit logs to address alert fatigue issues. The work in [10] extracted causal dependencies within alerts generated by EDR systems to reconstruct attacks. The authors of [32] collected causality logs from browsers' execution history to detect watering hole attacks. Additionally, the study [9] correlated provenance from individual hosts to create an enterprise-wide causal graph for locating APTs at the enterprise level. Correlation analysis is essential for fusing information from multiple devices or hosts.

However, this method faces challenges similar to those in data provenance, notably the dependence explosion [8], [9], for which no efficient solution exists. The explosion introduces bias and makes the final formed graphs challenging to analyze, or it consumes excessive resources, which is impractical in real detection scenarios.

c) Correlation Analysis: This analysis aims to uncover hidden relationships in large data sets from the perspective of co-occurrence. It involves two types of progressive abstract representations: frequent item sets, which define uncovered relationships and co-occurrence, and association rules, which define inter-dependence and conditional prior relationships. While frequent item sets measure coverage, association rules are evaluated based on support and confidence. The formulas to calculate support and confidence are:

$$\text{Support:} s(X \to Y) = P(X \cup Y) = \frac{\sigma(X \cup Y)}{N} \quad (1)$$

$$\text{Confidence:} c(X \to Y) = P(Y|X) = \frac{\sigma(X \cup Y)}{\sigma(X)} \quad (2)$$

Support s is used to eliminate uninteresting rules, and confidence c measures the reliability of the inference. However, association analysis does not imply causality, as it requires knowledge about causal and effect attributes [128].

In an effort to address cross-device tracking issues, the authors proposed the graph-based cross-device tracking framework GraphTrack in [129]. This framework leverages browsing histories among devices and models the complex interplays among IPs, domains, and devices to capture latent correlations. By employing a random walk with restart, they compute similarity scores between devices for tracking purposes. The framework's performance was evaluated on real-world datasets and compared with state-of-the-art methods, demonstrating superior results. However, the research solely relied on browsing histories and did not consider the impact of advanced tracking prevention mechanisms and clearing browsing history, which may degrade performance. Future research directions include defense through adversarial machine learning, studying real-world error distributions, and normalizing edge weights based on duration/interval.

POIROT [113] integrates both causality and correlation to support semantic analysis for detecting APTs over a long time span from existing systems' alerts. By mining causality between anomalous events, the system reconstructs alert chains and utilizes Latent Dirichlet Allocation (LDA) to model the semantic context of these chains. The LDA is a hierarchical Bayesian model that allows capturing latent attack intent and reconstructing APT scenarios. Testing on a real-world dataset yielded better precision rate compared to CONAN [80]. The system's limitations include the performance of LDA on short chain datasets. Future work focuses on real-time detection and response, implementation in other domains, and enhancing other security systems.

To detect complex watering hole attacks, which are common in the initial stage of APT attacks, the authors developed MNEMOSYNE [32], a postmortem forensic analysis engine. MNEMOSYNE accurately reconstructs, investigates, and assesses the ramifications of watering hole attacks with little modification to the browser. It collects causality logs related to browser execution and employs versioning techniques and user-level analysis to detect compromised websites and their modifications. The system surpasses detection solutions that rely on low-level semantics and offers a holistic internal forensic investigation of web-based watering hole attacks. However, it faces limitations concerning the current version of DevTools and domain-versioning technique dependence, which may reduce accuracy and potentially be orchestrated by adversaries. Additionally, investigating attacks relying on a drive-by download has limited visibility.

Although attack reconstruction technologies are prominent in the latest novel detection methods, some still rely on anomaly detection. Moreover, attack reconstruction typically filters alerts generated by anomaly detection systems.

2) Anomaly Detection: Anomaly detection is a fundamental concept employed by defenders to identify abnormal instances within original data. These anomalies can manifest as unusual data points that do not conform to the expected patterns or as outlier values that deviate significantly from the normal range. The definition of an anomaly can vary across different scenarios, thereby necessitating a diverse range of detection methods. In this study, we present a meticulous categorization of these methods, which results in the identification of four main groups.

a) Machine Learning Based Detection: The category of detection using machine learning involves the implementation of machine learning algorithms to detect outliers and anomalies through supervised or unsupervised learning. In some scenarios, ensembling different algorithms can enhance the robustness for clustering or classification. Several use cases for APT detection with machine learning algorithms are highlighted below.

To detect HTTP-based APT malware infection, [111] explored the usage of graph reasoning to build a web request graph using the referrer value in HTTP requests. They discovered that malware-related web access behaviors typically lack referrer values. By collecting referrer values, it becomes possible to distinguish between normal and malicious behaviors. The researchers conducted redirection reconstruc-

Covered Stages	reconnaissance	resource-development	initial-access	execution	persistence	privilege-escalation	defense-evasion	credential-access	ateral-movement	collection	exfiltration	command-and-control	impact
SteinerLog (2022) [8]	◗	◗	◗	◗	◗	◗	◗	◗	◗	◗	◗	◗	◗
TRACE (2021) [9]	◗	◗	◗	◗	◗	◗	◗	◗	◗	◗	◗	◗	◗
Hawk-Eye (2021) [6]	❍	❍	❍	❍	❍	❍	❍	❍	●	❍	●	❍	❍
ATLAS (2021) [7]	◗	◗	◗	◗	◗	◗	◗	◗	◗	◗	◗	◗	◗
RapSheet (2020) [10]	◗	◗	◗	◗	◗	◗	◗	◗	◗	◗	◗	◗	◗
Mnemosyne (2020) [32]	❍	❍	●	❍	❍	❍	❍	❍	❍	❍	❍	❍	❍
Hopper (2021) [109]	❍	❍	❍	❍	❍	❍	❍	❍	●	❍	❍	❍	❍
KCSM (2021) [17]	◗	◗	◗	◗	◗	◗	◗	◗	◗	◗	◗	◗	◗
CONAN (2022) [80]	❍	❍	●	❍	❍	❍	❍	❍	❍	❍	●	❍	❍
Euler (2023) [110]	❍	❍	❍	❍	❍	❍	❍	❍	●	❍	❍	❍	❍
Niu W. (2021) [111]	❍	❍	❍	❍	❍	❍	❍	❍	❍	❍	●	❍	❍
Yan G. (2020) [112]	❍	❍	❍	❍	❍	❍	❍	❍	❍	❍	◗	❍	❍
Poirot (2022) [113]	◗	◗	◗	◗	◗	◗	◗	◗	◗	◗	◗	◗	◗

TABLE VI: Covered Stages

●:certainly covered ◗:possibly covered ❍: not covered

tion and URL similarity optimization to create the request graph. Utilizing local outlier factor-based anomaly detection, they filtered out normal traffic, leaving behind anomalies. Their proposed solution outperformed rule-based detection by automatically correcting normal HTTP requests without specific rules, and it also outperformed machine learning-based detection by exploring the correction of referred and requested URLs without requiring a large amount of labeled data. However, their methodology does have limitations in detecting forged referrer values and detecting secret channels. Their future plans involve focusing on behavioral detection and packet transmission.

For efficient detection of advanced attacks in a software-defined networking (SDN) environment, [130] introduced a novel multi-module machine learning (ML) framework. It combines an optimized feature selection schema with advanced unsupervised machine learning techniques to improve detection performance while reducing computational complexity. They developed a novel data flow collection (DFC) module for scalable feature gathering from network data using the sFlow protocol, which adopts mandatory sampling for scalability. The information gain was utilized for feature selection, and the unsupervised Isolation Forest algorithm was employed for network security threat detection. This work outperformed state-of-the-art contributions in accuracy and detection while significantly reducing computation complexity. It addresses the limitations of existing attack detection schemes, such as high false positive rates, low detection performances, and high computational costs. However, the experiments were conducted only on a single dataset, which may not be representative of all possible APT scenarios in IoT networks. Additionally, reliance on the sFlow protocol may not be supported by all network devices. Nevertheless, this work serves as a foundation for developing more advanced security solutions and can incorporate other unsupervised ML techniques to enhance the overall security of SDN environments.

Another study [131] also utilized multiple machine learning techniques to enhance APT detection in Internet of Things (IoT) settings. They proposed a prior knowledge input (PKI) model, wherein an unsupervised clustering method was applied for pre-classifying the original dataset. The extracted prior knowledge was incorporated into the supervised models to reduce training complexity and determine the optimal mapping between the raw data and true labels. Three supervised learning algorithms (decision tree, random forest, and XGBoost) formed the baseline model, and this work outperformed the baseline model in various measurement metrics. However, the testing was conducted on a single laboratory-created dataset, which may not represent all possible APT scenarios in IoT networks. Additionally, practicality and scalability in large-scale IoT networks may require further consideration.

While pure machine learning-based solutions suffer from the inability of continuous learning and adaptation, deep learning-related models have outperformed in more complex situations with continuous ability. The trend follows consistently updated new methods and exploitations in the field.

b) Deep Learning Based Detection: Deep learning is a specialized area of machine learning that involves training artificial neural networks to analyze vast amounts of data and make predictions. These neural networks consist of interconnected nodes, called neurons, arranged in multiple layers. The networks are designed to automatically learn hierarchical representations of the data through feature learning or representation learning. In the context of cybersecurity, deep learning is widely used for learning feature spaces and predicting whether activities or behaviors are malicious. Numerous works have explored the application of deep learning in detecting APT activities.

To address enterprise-level lateral movement detection, a study by King et al. proposed a scalable temporal link prediction system called Euler for anomaly-based intrusion detection [110]. This approach integrates graph neural networks (GNNs)

Data Source	alerts	audit logs	network traffic	system logs	application logs	browser history	common logs	event logs	network snapshots	http traffic	dns logs	binary	threat reports	string, bytecode	devices	package	functional libraries	pre-trained models
SteinerLog (2022) [8]	✓	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×
TRACE (2021) [9]	×	✓	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×
Hawk-Eye (2021) [6]	×	×	✓	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×
ATLAS (2021) [7]	×	×	×	✓	✓	×	×	×	×	×	×	×	×	×	×	×	×	×
RapSheet (2020) [10]	✓	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×
Mnemosyne (2020) [32]	×	×	×	×	×	✓	×	×	×	×	×	×	×	×	×	×	×	×
Hopper (2021) [109]	×	×	×	×	×	×	✓	×	×	×	×	×	×	×	×	×	×	×
KCSM (2021) [17]	✓	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×
CONAN (2022) [80]	×	×	×	×	×	×	×	✓	×	×	×	×	×	×	×	×	×	×
Euler (2023) [110]	×	×	×	×	×	×	×	×	✓	×	×	×	×	×	×	×	×	×
Niu W. (2021) [111]	×	×	×	×	×	×	×	×	×	✓	×	×	×	×	×	×	×	×
Yan G. (2020) [112]	×	×	×	×	×	×	×	×	×	×	✓	×	×	×	×	×	×	×
Poirot (2022) [113]	✓	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×
Exorcist (2022) [5]	×	×	×	×	×	×	×	×	×	×	×	✓	×	×	×	×	×	×
yeboah (2019) [12]	×	×	×	×	×	×	×	×	×	×	×	×	✓	×	×	×	×	×
Jiang W. (2022) [59]	×	×	×	×	×	×	×	×	×	×	×	×	✓	×	×	×	×	✓
Ladisa (2022) [13]	×	×	×	×	×	×	×	×	×	×	×	×	×	✓	×	×	×	×
Eggers (2021) [22]	×	×	×	×	×	×	×	×	×	×	×	×	×	×	✓	×	×	×
MALOSS (2020) [14]	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	✓	×	×
HARP (2021) [15]	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	✓	×
SGX (2020) [46]	×	×	×	×	×	×	×	×	×	×	×	✓	×	×	×	×	×	×

TABLE VII: Collection of Data Source in Academic Detection

✓:used ×:not used

and recurrent neural networks (RNNs) using an encoder-decoder structure in a distributed manner. GNNs map network snapshots to low-dimensional embedding vectors, while RNNs encode the dynamic changes between connections. A leader machine controls the workflow and instructs multiple workers responsible for parallel processing of network snapshots. This approach is effective in distinguishing dynamic network changes in different timestamps, offering an advantage over solutions focusing on static network status. Future directions include testing other topological or temporal encoders, improving classifiers, and evaluating the solution on large-scale temporal graph datasets.

To uncover hidden APT behaviors, Yan et al. explored DNS activities for detection by leveraging features between DNS requests and responses [112]. They devised novel features based on time intervals of requests and responses and host frequency analysis, which, when combined with other APT-related DNS features, resulted in robust detection using a custom deep neural network. Comparison experiments demonstrated that the proposed features enhanced the detection rate of other detection systems. The long-term data collection capability allowed analysis of DNS behaviors over an extended period, unlike solutions relying on short-term datasets. However, the proposed features potentially increased the suspicion level of some white traffic due to similar time interval features. Addressing this drawback is the researchers' next research goal, along with introducing a feedback mechanism for model updates.

To preserve privacy while employing proactive defense approaches, Naseri et al. explored the feasibility of using Federated Learning (FL) to predict future security events [132]. They developed the CERBERUS system, which adopts Recurrent Neural Network (RNN) collaboratively from different organizations. The main objective was to achieve privacy while making slight accuracy sacrifices. The system underwent utility, robustness, and privacy testing through comparison experiments on non-independent-and-identically-distributed (non-IDD) datasets, as well as adversary attacks and defense on models. Central differential privacy (CPD) in FL was applied to defend against inference attacks, achieved by sharing only dataset information. Additionally, algorithms were created to calculate Non-IIDness score and participant's contribution impact. Performance varied across benefits to participants, model accuracy compared to central solution, and tuning of False Positives Rates (FPRs). Improving overall response to alerts and accuracy remains an area for improvement in the current system.

Despite the effectiveness of deep learning solutions, they suffer from interpretability issues, being akin to black-box models [21], [133]. Bias may be introduced due to data quality and uncontrolled factors [10], leading to false positives in deep learning-based anomaly detection systems. To address potential bias and false positives, other solutions using predefined rules and scoring-based anomalies have been proposed in various works.

c) Attack Specification: The attack specification is a rule-based detection method that defines specific rules based on empirical findings about various attacks, often referred to as Indicators of Compromise (IOC) or Indicator of Attack (IOA) in industrial contexts [8], [137]. In the academic domain, the

Name	Detection Opportunities
SolarWinds [56], [57]	config in SSL certificates, IP address and logon activity, certain patterns of SMB sessions
CCleaner [134]	DGA detection, abnormal behaviour, execution of shellcode
NotPetya [93], [135]	special process execution, network propagation with SMB, unauthorized MBR modification
Kaseya VSA [94], [95]	certain hashes of files and domains
Cozy Bear [68], [69]	system binary proxy execution, logon auto-start execution
OilRig [136]	scheduled tasks, DNS protocol
Lazarus Group [73], [74]	DLL side-loading, reflective DLL injection, DLL search order hijacking
Codecov [97], [98]	dependencies, atypical IP addresses or regions, abnormal usage of env variables

TABLE VIII: Collection of Industrial Detection Methods against Supply Chain based APT

work by Alageel et al. [6] introduced the HAWK-EYE system, which aims to achieve holistic detection for APT C2 domains. By generating novel features from semantic and structural properties of domains using DNS infrastructure, the system can classify domain names and evaluate the performance of various features. It serves as a parallel or first filter role for network intrusion detection systems (NIDS) and enhances the detection ability with new features. However, it has some limitations, such as high assumed False Positive Rates (FPRs), bias in data collection experiments without considering legitimate host features, and a lack of fuzzy matching in the implementation of uni-gram domain matching.

To effectively detect the attack in lateral movement stage, the work [109] proposed the system Hopper that uses common-collected log data to detect lateral movement attack with a manageable rate of false alarms. The lateral movement is typically a crucial stage of an APT attack. The system leveraged an attack specification that captures fundamental characteristics of lateral movement as a set of key path properties. which has been integrated into the anomaly detection algorithm. This rule-based anomaly scoring algorithm can surface the login paths that most likely reflect lateral movement via reconstructed global movement activity from point-wise login events. Even through this work has low false positive rate, the system can still be evaded if the adversary can piggyback on top of a series of logins made by legitimate users to access their target machines or just install stealthy malware on a shared server that lies on the path to the target machines. Therefore, the future research directions focus on defence of evasion attack, expansion of the generality with other network architectures and improvement on performance.

The approach presented in [17], focusing on APT detection and addressing alert fatigue from a network direction perspective, seems promising. The network direction points at the movement of the attack activity between network zones. They proposed the APT scenario graphs derived from alerts. The algorithm to generate APT scenario graphs contains a series of sequential steps. By firstly defining the network direction inside network zones according to Network kill chain state machine (NKCSM) stages, a single APT stage is assigned to each alert and meta-alert. Then the alert graph is generated by following rules like timelines and latest infections. After that, the APT infection graph is derived by aggregating existing information and eliminating obsolete information, which can massively reduce the graph density. Finally, the APT scenario graph is created by further extracting valid paths between stages. Their work is out of state-of-the-art performance on reduction of alert volume using alert correlation and attack contextualization. Their next plans are to integrate host-level and user identify context for richer scenario graphs and also eliminate false positives.

Furthermore, attack specification designs have been integrated into other detection systems. CONAN [80] proposed the concept of atomic suspicious indicators (ASIs) and incorporated them with rules and malicious states as configuration files in their built detection system. This integration achieved near zero false positives, as explained in their research.

d) Scoring Algorithm: The scoring algorithm is a common component found in various solutions. It typically involves two types of scores: similarity scores and anomaly scores. Anomaly scores are used to aggregate anomaly or risk scores in most cases. On the other hand, similarity scores can be computed at the code-level or path-level, as explored in the work by Wang et al. [129], where they tackled the tracking problem among devices and calculated similarity scores between devices for tracking purposes. Similarly, Yi et al. [62] compared code in forked projects with their original counterparts to identify the spread of vulnerabilities using similarity score computation.

In contrast, anomaly scores find broader application. Ho et al. [109] utilized a rule-based anomaly scoring algorithm to identify anomaly paths when locating APTs during the lateral movement stage. Another study by Hassan et al. [10] introduced a threat scoring methodology that enhances accuracy and performance in APT detection. Bhattarai et al. [8] defined risk scores based on different events in various network layers, computed using rarity scores and sparsity scores of anomaly events. Additionally, intelligence-driven detection algorithms, proposed by Li et al. [21], employed risk scoring to identify anomalies based on risk thresholds.

B. Censoring Methods

This category encompasses defense methods specifically targeting supply chain attacks, as presented in Table V based on Yang's Poirot [113]. The focus of these works is primarily on early detection at the point of infection, with an emphasis on code-level analysis. We have further divided this category into three subgroups to provide a more detailed classification of these defense methods.

1) Differential Analysis: Differential analysis is a methodical and comprehensive investigation that involves comparing and contrasting various elements, variables, or scenarios to discern patterns, discrepancies, or alterations among them.

Data Source	scanning	login activities	scheduling tasks	SMB logs	network traffic	execution flow	system logs	execution logs	event logs	registry	DLL, PE file events	code artifacts	redirection files	audit logs	authentication logs
SolarWinds [56], [57]	✓	✓	✓	✓	×	×	×	×	×	×	×	×	×	×	×
CCleaner [134]	×	×	×	×	✓	✓	×	×	×	×	×	×	×	×	×
NotPetya [93], [135]	×	×	×	×	✓	✓	✓	×	×	×	×	×	×	×	×
Kaseya VSA [94], [95]	×	×	×	×	×	✓	×	×	✓	×	×	×	×	×	×
Cozy Bear [68], [69]	×	×	×	×	×	✓	×	×	×	✓	×	×	×	×	×
OilRig [136]	✓	×	×	×	✓	×	×	×	×	×	×	×	×	×	×
Lazarus Group [73], [74]	×	×	×	×	×	×	×	×	×	×	✓	✓	✓	×	×
Codecov [97], [98]	×	×	×	×	✓	×	×	✓	×	×	×	×	×	✓	✓

TABLE IX: Collection of Data Source in Industrial Detection

✓: used ×: not used

By systematically evaluating and contrasting the characteristics, behaviors, or outcomes of two or more subjects or situations, researchers aim to gain insight into the factors that contribute to their dissimilarities or similarities. In the context of software security, conducting a comparison between benign versions and infected versions of software is crucial for detecting potential malicious components. This process aids in identifying and isolating the specific areas of divergence or modification, enabling the identification of potential threats and vulnerabilities. Through differential analysis, researchers can effectively discern anomalies or irregularities that may indicate the presence of malicious code or tampering within the software's source code or executable files. The systematic examination of variations between different software versions is instrumental in enhancing the overall security and integrity of software supply chains.

The work [5] focuses on APT attack targeting supply chains via closed-source software. The researchers deployed an approach to detecting the insertion of malicious functionality in supply chain through differential analysis of binaries. The differential analysis points at the comparison of binary analysis between build versions instead of analyzing a single version. By building the system Exorcist to automatically analyse two examples, the malicious indicators can be found via static analysis, dynamic analysis and binary analysis. The outcome of analysed results are weighted in order to increase the reliability. The project is limited by specific research target of Windows Portable Executable (PE) files instead of covering other OS types. Other than that, the scope is controlled to be infected binaries during the supply chain; they excluded the dynamic behavior for Dynamic-link libraries (dlls) and also the direct analysis for the code preceding its compilation into an executable.

2) Program Analysis: Program analysis in this context refers to the examination of code at the code-level, encompassing both static and dynamic analysis techniques. Static analysis involves scrutinizing the source code or program's structure without executing it. It aims to identify potential issues, vulnerabilities, or code patterns that could lead to security risks. On the other hand, dynamic analysis focuses on monitoring the program's behavior during its run-time execution. It involves observing and recording various aspects, such as function calls, memory usage, input/output operations, and interactions with external resources. By observing the program in action, dynamic analysis can provide valuable insights into its actual behavior and potential runtime vulnerabilities that may not be evident through static analysis alone. Together, these two approaches complement each other and contribute to a comprehensive understanding of the program's security posture and potential risks.

The research conducted in [13] focuses on the supply chain of the Java ecosystem, driven by the scarcity of software supply chain studies in this specific domain. The authors proposed the static analysis approaches for malicious injected OSS Java repositories. The static solution focuses on the features extracted from constant pool and bytecode instructions with intra-procedural data-flow analysis. By extracting critical information, their solution achieved promising detection rate with less amount of data to review. The detection rate is far higher than the low detection within antivirus (AV) methods. Whereas, the research has wide limitations from false positives, restricted dataset, restricted malware types, limited complexity of payload and vulnerable entropy analysis method. They are going to improve data-flow analysis with inter-procedural flows, characterise maven central ecosystem for critical API calls, and apply indicators in automatic classification approaches in anomaly detection.

In [62], the authors present an alternative approach to identifying propagated vulnerabilities in forked blockchain programs through the development of a tool called BlockScope. The analysis is based on patched code in original projects. By extracting the key statements (ks) in patch code via grep [16] function, which is a tool to search specific string inside a program or file, it is possible to find the boundary of the ks by searching surrounding lines of code. According to the similarity computation algorithm they defined, it is feasible to justify whether the new code is vulnerable or not by comparing the computed similarity score with a pre-defined

[16]https://man7.org/linux/man-pages/man1/grep.1.html

average threshold. The similarity algorithm was created mainly based on the normalized Levenshtein distance [138]. In addition, during the searching process, the developed tool can automatically extract and leverage the context of patch code to narrow down the search scope, which is quite efficient. Their works have been confirmed by communities through reports of found vulnerabilities. However, limited by the research targets, the study may potentially uncover vulnerabilities in other blockchain projects and also unknown vulnerabilities in forked projects. The next plan is to extend the scope to other non-blockchain domains and improve the generality.

3) Machine Learning:

a) Active Learning: To address vulnerabilities in Open Source Software (OSS), Vasilakis et al. [15] proposed a novel technique called Active Library Learning and Regeneration (ALR) for inferring and regenerating the client-observable behavior of software components. ALR is the core technique integrated into the created system Harp which is focusing on string processing libraries. It is consist of several stages including exploration, inference and regeneration. In exploration, ALR uses increasingly sophisticated rounds of exploration to generate inputs, provides these inputs to the component, and observes the resulting outputs to infer a model of the component's behaviour as a program in a domain-specific language (DSL). The DSL can promote the inference and representation process of the library behaviours and remedy the malicious computation. During regeneration, ALR can regenerate the client-observable behaviours of the component by using the model to generate code which is equal to the original one while free of vulnerabilities. Apart from those techniques, Harp has also synthesized the static and dynamic analysis to infer the behaviour of string processing components.

Despite of above strengths, this research suffers from limitations on four aspects. Firstly, there is a concern regarding synthesis, primarily stemming from the inability to provide desirable guarantees such as infinite running time in specific cases. Another drawback is the system's domain-specificity, as it was exclusively designed for string-processing components, neglecting other types of components. Additionally, there remains a concern about performance overhead, and the evaluation has been somewhat limited. As a result, the researchers aim to extend current approaches to encompass other types of software components.

C. Proactive Methods

The proactive method involves deploying defense technologies before the occurrence of APT attacks. This approach can be approached from distinct angles. Firstly, it begins with assessing an institution's assets and identifying potential attack vectors targeting those assets. This includes collecting and analyzing public threat data and implementing typical defense solutions. These actions fall under the umbrella of risk management, encompassing impact analysis for owned code or projects, enumeration of attack vectors, and the utilization of cyber intelligence.

Another perspective of the proactive method focuses on prevention. One of the most efficient strategies found is the application of blockchain technology to ensure the verification of official digital sources or provenance data. By leveraging blockchain's immutable and transparent nature, organizations can enhance the trustworthiness and integrity of their digital assets, thereby mitigating the risks associated with potential APT attacks.

1) Risk Management:

a) Impact Analysis: In [16], the number of download has been mentioned as one of the attributes to measure the impact of influence of an OSS attack. Similarly, in [59], the researchers firstly explored the subset of software supply chains with Pre-trained Model (PTM) Supply Chain. The PTMs point at models that have been trained and stored at model hubs like Hugging Face [17], in which the popular models usually have millions of downloads. The authors synthesized techniques from traditional package ecosystem security studies and deep learning attacks to analyse potential threat models of PTM supply chain. They applied a combination of quantitative and qualitative methods to measure the potential security risks via empirical study of artifacts and risks in eight popular model hubs. As a result, they indicated that the existing defenses are insufficient for ensuring the security of PTMs. They proposed further directions including empirical study side for dependencies analysis and impact analysis of influencing maintainers, and the development of automated tool for model audit and model scanning.

From the report [40] released by anyLogistix [18], which is a well-known supply chain software simulation company, the risks have been split into operational and disruptive risks. Once a disruption happens, its effects tend to propagate down the supply chain, which is also called ripple effect. The long-term cyber-attack is generally one type of disruptive risks. Through the simulation, optimization and data analytic against the risks, it is possible to reduce the impact by strategic planning. Another industrial solution from NIST [139] is the impact analysis tool, which measures the relative impact of potential supply chain disruptions. The users can benefit from that to identify highly impactful and interdependent nodes where focuses risk-mitigating controls can be implemented.

In [140], a risk management approach was proposed to address the APT response problem. With the goal of mitigating potential loss for an organization, this work modeled the evaluation of expected loss per unit time, limited expected state, and response resources with graphic topology structures of interactive hosts inside the organization. This model is a type of game-theoretic problem in which the organization seeks a Nash equilibrium. The authors presented a greedy algorithm for solving the game and recommended the response strategy generated by performing the algorithm. To verify the efficiency, the authors designed multiple attack/response strategies to assess the resulting potential loss. Finally, they concluded the optimized loss under various situations, considering different aspects of information like potential loss, value vector, network topology, or targeted organization.

[17] https://huggingface.co/
[18] https://www.anylogistix.com/

b) Attack Vector Enumeration: To measure the impact of supply chain attacks on package managers, [14] proposed a comparative framework to qualitatively assess the functional and security features of package managers. The authors analyzed numerous package managers for interpreted languages using well-known program analysis techniques, including metadata analysis, automatic, and static analysis. Through iterative verification, they discovered various malicious programs in PyPI, NPM, and RubyGems [55]. While their research aimed to gain insights from existing attacks and identify new ones, it may not have achieved high coverage and robustness in program analysis yet. Moreover, the vetting pipeline currently suffers from inaccuracy in static analysis and low coverage in dynamic analysis. Additionally, the focus was limited to interpreted languages, and comprehensive solutions to the problem of supply chain attacks were not provided. Their future plans include improving the accuracy and coverage of program analysis techniques and exploring more effective vetting pipelines.

In [22], the cyber-attack surface related to nuclear-related supply chains was investigated. The authors discussed overlooked supply chain threats and vulnerabilities and designed an attack surface diagram to enumerate risks and potential attacks between different steps. The diagram aids security personnel in enhancing risk management processes. This research serves as a platform for further supply chain research and can help develop supply chains with provenance capabilities. It also motivates the development of improved security credential verification processes and secure tamper-proof distribution methods.

Due to the complex inter-dependencies among numerous supply chain stakeholders, challenges arise, including the lack of third-party audit schemes and cascading cyber threats. In [18], the authors modeled and analyzed CSC attacks and cyber threats within the supply chain. They integrated concepts from threat intelligence and industrial practices like the STIX threat model to conduct a case study of an Electric Power Station, aiming to determine attack propagation and cascading effects. Employing Common Vulnerability Scoring System (CVSS) concepts, they parameterized and quantified different elements such as Penetration and Manipulation to calculate the probability of successful random distribution, thereby determining the severity of attacks. Future studies involve modeling power grid sub-station systems and determining cybercrime risks in cyber-physical systems, along with detailed threat analysis using machine learning techniques.

c) Cyber Intelligence: In the talk presented at Black Hat USA 2019, titled "Death to the IOC," researchers from Microsoft Defense Team demonstrated their approaches to extracting Tactics, Techniques, and Procedures (TTPs) from reports and public data using name entity recognition, a type of Natural Language Processing (NLP) technology that extracts specific tokens from text-based data. By visualizing the relationships between various APTs, the extracted data contributed to optimizing the deployment of defense mechanisms. Additionally, the Indicators of Compromise (IOCs) within the data were utilized as detection clues. Other works such as [18], [8], and [141] have also integrated cyber or threat intelligence as part of data sources for detection systems.

In the project of [21], the authors utilized cyber intelligence collected from devices to locate APTs targeting the edge of the network. They developed an edge Bayesian Stackelberg game and a cyber threat intelligence (CTI) based defense strategy model for APT detection. The Stackelberg game is a theory in game theory that involves different actions on the marketing price between leader and follower firms. The authors created a dynamic resource allocation algorithm inspired by deep reinforcement learning (DRL)-based resource allocation schema, which balanced the trade-off between detection accuracy and computational resources needed. Another algorithm developed by them was the intelligence-driven real-time detection algorithm, which was based on a sliding window and threat level to generate features. Anomalies were identified through comparison with pre-set thresholds. Their work optimized the detection model and addressed the limitations of existing mechanisms, such as lack of transparency and performance in the detection model. The next step for this project is to extend to distributed collaborative defense framework under the premise of privacy. To achieve that, the primacy solution it to integrate federated learning and privacy computation technologies to establish encrypted shared defense models between edge devices and servers.

2) Block-Chain Based Design: In order to address potential attacks targeting the CSC, Bandara et al. [142] proposed a system called Let'sTrace. This system integrated blockchain, the update framework (TUF), In-Toto frameworks, and federated learning (FL) to enhance CSC security. The TUF provides protection for update systems, while the In-Toto framework ensures a secure software development life cycle. Through a hierarchical design, the authors implemented blockchain between different stakeholders and employed smart contracts among nodes to provide verification and authentication for identities and transmitted messages. The TUF and In-Toto layers further verified the integrity of updates and supply chain information. Leveraging FL, the system conducted further analysis of update information and supply chain data in a private manner. These solutions significantly reduced the attack vectors related to signing keys, compromised repositories, third-party software, and vulnerable update mechanisms. The authors' future plan involves adopting zero trust security principles to address CSC attacks.

In the work of [46], another possible solution to prevent the stealing of signatures in hardware development is the deployment of a blockchain infrastructure with smart contracts. This approach verifies trusted counterparts during integration by using encryption methods with public and private keys shared between senders and receivers.

VI. Challenges and Research Opportunities

The authors in [20] conducted a comprehensive exploration of the challenges and proposed future research directions in detecting APT attacks. They aimed to identify the limitations of existing research, particularly in areas that were not adequately covered. Through their survey, they discovered that most detection methods focused on limited stages of

Proposed Challenges	limited stages detection	computation overhead	false positives	restricted precision	in-time update	scalability	entropy analysis	uncertainty	finite running	long-time attack campaign	unknown attacks	supply chain attacks	evasion	sole data source	sole group of features	comparative features/data	sole system	novel exploitation methods
SteinerLog (2022) [8]	×	×	×	×	×	×	×	×	×	✓	✓	✓	✓	×	×	×	×	×
TRACE (2021) [9]	✓	✓	×	×	×	×	×	×	×	×	×	✓	✓	×	×	×	×	×
Hawk-Eye (2021) [6]	×	×	✓	×	×	×	×	×	×	×	×	✓	✓	×	×	✓	×	×
ATLAS (2021) [7]	×	×	×	✓	×	×	×	×	×	×	×	✓	✓	✓	×	×	×	×
RapSheet (2020) [10]	×	×	×	×	×	×	×	×	×	×	×	✓	✓	✓	×	×	×	×
Mnemosyne (2020) [32]	×	×	×	×	×	×	×	×	×	×	×	✓	✓	×	×	×	×	✓
Hopper (2021) [109]	×	×	×	×	×	×	×	×	×	×	✓	×	×	×	×	×	×	×
KCSM (2021) [17]	×	×	✓	×	×	×	×	×	×	×	×	✓	✓	✓	×	×	×	×
Euler (2023) [110]	✓	×	×	×	×	×	×	×	×	×	×	✓	✓	×	×	×	×	×
Niu W. (2021) [111]	×	×	×	×	×	×	×	×	×	×	×	✓	✓	×	×	×	×	×
Yan G. (2020) [112]	×	×	×	×	✓	×	×	×	×	×	×	✓	✓	×	×	✓	×	×
CONAN (2022) [80]	×	×	×	×	×	×	×	×	×	×	✓	✓	✓	×	×	×	×	×
Exorcist (2022) [5]	×	×	×	×	×	×	×	×	×	×	×	×	×	×	✓	×	✓	×
yeboah (2019) [12]	×	✓	×	×	×	✓	×	×	×	×	✓	×	✓	×	×	×	✓	×
Ladisa (2022) [13]	×	×	×	×	×	×	✓	✓	×	×	×	×	✓	×	×	×	×	✓
HARP (2021) [15]	×	✓	×	×	×	×	×	×	✓	×	×	×	✓	×	✓	×	×	×

TABLE X: Proposed Challenges in References

✓: mentioned ×: not mentioned

APT attacks, often excluding the entire attack process. They discussed the challenges posed by determined attacks, the prolonged duration of attacks, as well as internal employees and infrastructure-based challenges. Based on these identified challenges, the authors proposed potential opportunities, including full-stage APT detection, automatic detection and correlation of spear-phishing emails, detection of fileless malware considering factors like long time frames, model stealth, and reverse engineering, exploration of dark-web forums, and APT defense in cloud computing environments.

Throughout our investigation, we delve into the latest research within the specific sub-direction of exploiting SCVs. As outlined in Table V, we find that recent works have made strides in addressing certain potential challenges. Notably, although some works may not explicitly mention full-stage APT detection, they demonstrate the capability to detect and track entire APT attack stages [7]–[10], [17]. However, it's essential to acknowledge that in practical scenarios, achieving full-stage detection may introduce biases and side effects on performance.

In an effort to offer an updated understanding of the current research challenges, we have meticulously documented the limitations and uncovered aspects, as illustrated in Table X. Enriching the context, we have considered both the supply chain and evasion attack perspectives. The supply chain aspect contributes to addressing typical types of APT attacks, while evasion attacks focus on potential adversarial attacks for data falsification, which can undermine defense systems. It's noteworthy that some projects have discussed potential adversarial or evasion attacks but did not implement corresponding mechanisms in their systems or tools [5], [12], [13], [15], [59]. In the following discussion, we present critical challenges drawn from the investigated papers:

Long-Term APTs: In the simulated experimental environment, the APT campaigns were designed to span almost one month [8]. However, real-world APT activities can extend over several months [8], presenting challenges not only in data storage but also in analytical difficulties [9], [20]. The longevity of APT behaviors introduces common challenges, such as the explosion of dependencies and the considerable growth in the size of provenance graphs during causality and provenance analysis [80]. Moreover, malicious attackers often remain dormant for extended periods, sometimes lasting for weeks [57], hindering detection based on short-term historical or sequential data [7]. Techniques employed to reduce graph complexity in data provenance-based solutions [8]–[10] may potentially impede the discovery of critical information for these sophisticated attacks.

False Positives: Many methodologies or systems based on causality analysis operate above alerts generated by anomaly detection systems, assuming the high quality of these alerts. For instance, works such as [8], [10], [17], [113] (refer to Table VII) heavily depend on anomaly-generated alerts as their primary data source. The efficacy of these methodologies is consequently tethered to the accuracy of anomaly detection systems. While causality or provenance technologies may help mitigate false positives, the precision of the initial alerts remains crucial. Other methods relying on network, host, or browser-level logs also grapple with relatively high false-positive challenges [6], [132].

Single Detection Source Data: Previous works in the field have predominantly depended on a single data source, such as browser-based history, audit logs, or network traffic, as the cornerstone of their data provenance-based approaches [7],

Direction	Opportunities
performance	provenance optimization on noise reduction of irrelevant information
	causal graph optimization on computation, storage and dependency explosion
data fusion	multi-source data fusion / correlation
novel features & attacks	extract novel features from more stealthy attack scenarios
	extract features from supply chain attack that can be used for detection
robustness	consider evasion attack from adversarial examples
	conceal attack target models or data pool
privacy	consider the privacy of whole data collection and also training stages
accuracy and bias	find solution to solve dependency exploitation when using provenance
	introduce credible scoring mechanisms when building attack scenarios
	overcome the challenge from long-time attack scenario with efficient solutions
	overcome the bias from attack-specification rules and common attack chain

TABLE XI: Research Opportunities

[17]. Table VII illustrates the prevalence of using a singular type of data in academic research to detect or locate malicious behaviors. While some efforts, like [7], have integrated two data sources, such as system and application logs in Windows systems, these sources still fall into the same category of low-level logs, potentially lacking insights into higher-level behaviors. Moreover, approaches based on a single data source may prove ineffective against camouflage and encryption techniques [80], limiting their overall detection capabilities. In contrast, industrial solutions, as depicted in Table IX, frequently harness multiple evidence sources to establish a more comprehensive understanding of specific technologies for mitigation and defense purposes.

Lack of Holistic Detection on Comprehensive Stages: Our investigation, outlined in Table VI, highlights a notable gap in existing research—few studies provide a comprehensive coverage of all stages of an APT lifecycle [20]. The intricacies of correlation and the limited availability of data for investigation pose significant challenges in achieving a holistic understanding of APT activities. Furthermore, the diverse attack vectors exploited by APT actors across different stages add complexity, making it difficult to encompass all aspects for early detection. However, it's worth noting that early detection doesn't necessarily require analyzing every phase of an APT attack. An intriguing avenue for future research involves identifying critical stages that maximize exposure to APT behavior, considering computational and analytical overhead. Some initiatives have already started exploring this direction, such as [80].

Evasion and Privacy: While numerous efficient systems demonstrate remarkable detection performance, some still lack robust defenses against potential evasion attacks. Threats such as adversarial data, model stealing, and privacy disclosure can pose serious challenges [8], [109], [129]. Certain initiatives have focused on addressing privacy and robustness issues [132], [142]. Privacy protection remains a crucial concern, especially given the increasing use of artificial intelligence (AI) in recent years.

Supply Chain: New technology introduction and evaluation in the supply chain introduces new challenges concerning cyber risks. The adoption of technologies such as Blockchain, Smart Contracts, AI applications, Internet of Things (IoTs), and Cyber-Physical Systems in Supply Chain 4.0 enhances industry efficiency and improves people's lives [11], [142]. However, the integration of different devices, vendors, or services within the supply chain creates potential attack surfaces for adversaries. Limited access to data and a lack of control over third parties make it challenging to conduct thorough investigations before integration, introducing new cyber risks [15], [16], [18].

Low Detection for CSC Attack: Many research works focusing on supply chain attacks concentrate on detecting vulnerabilities in third-party vendors or software during the initial stage [5], [13], [22], [61]. However, practical considerations, such as policy requirements, privacy concerns, copyright issues, and data availability, often impose restrictions on the analysis in this area. Since supply chain-based APTs constitute a novel attack type, historical data or pattern-based solutions [7], [9], [17], [109] may become outdated and unable to detect new attacks. Ensuring the feasibility of detecting these concealed attacks at an early stage is essential for ensuring the safety of the entire supply chain.

Stealthiness: Practical reviews highlight a significant challenge in detecting certain APTs that manage to remain undetected for extended periods. Many existing detection and monitoring solutions struggle to identify these threats. For instance, APTs exploiting SCVs, as discussed in [5], benefit from inherent stealthiness due to the default trust placed in third-party suppliers. Additionally, some APTs exploit novel communication challenges [27] or leverage information-hiding technologies [34], [61], [131], making their detection a formidable task. Unveiling APTs employing highly stealthy techniques presents an area for further exploration.

In addition, we present a table (Table XI) that groups potential research opportunities based on the challenges discussed earlier. In this table, we categorize further opportunities into six perspectives, covering stages from data collection and processing to detection performance, efficiency, and the final robustness and privacy of detection systems.

VII. Conclusion

In this survey, we focus on a special type of APT that exploits SCVs, a subject that, to the best of our knowledge, has not been thoroughly researched before. We gather real-world APT attack scenarios and analyze both industrial detection and academic defense methods to provide a comprehensive overview of the current challenges, solutions, and future directions in this domain. Employing various analysis techniques such as graph-based analysis, statistical analysis, and comparative studies, we extract valuable insights from the collected

data. To better comprehend the current threat landscape, we present a triage of supply chain-based threats based on different frameworks. Additionally, we propose a unified threat model based on actual scenarios and develop a customized detection chain. By combining hardware and software supply chain perspectives, we conduct an in-depth enumeration of attack vectors, offering guidance for defense deployment. To showcase state-of-the-art defense methodologies, we further classify defense technologies pertaining to both APTs and supply chain attacks. Finally, we identify limitations and drawbacks from existing literature, elucidating the challenges and outlining future research directions. Our intention is for this survey to benefit both industry and academia and contribute to safeguarding the Internet world against APTs exploiting SCVs.

Acknowledgment

This work is partially funded by the Horizon Europe grant 'Integration & Harmonization of Logistic Operations' (TRACE) #101104278.

References

[1] Mandiant, "Mandiant m-trends 2023," 2023, [Online: Accessed 3 June 2023]. [Online]. Available: https://www.mandiant.com/resources/blog/m-trends-2023

[2] OWASP.org, "Owasp top 10 for llm applications," 2023. [Online]. Available: https://owasp.org/www-project-top-10-for-large-language-model-applications/assets/PDF/OWASP-Top-10-for-LLMs-2023-v1_0_1.pdf

[3] European Union Agency for Cybersecurity., *ENISA Threat Landscape for Supply Chain Attacks.* LU: Publications Office, 2021.

[4] S. Ozarslan, "Tactics, techniques, and procedures (ttps) used in the solarwinds breach," 2020, [Online: Accessed 5 July 2023]. [Online]. Available: https://www.picussecurity.com/resource/blog/ttps-used-in-the-solarwinds-breach

[5] F. Barr-Smith, T. Blazytko, R. Baker, and I. Martinovic, "Exorcist: Automated Differential Analysis to Detect Compromises in Closed-Source Software Supply Chains," in *Proceedings of the 2022 ACM Workshop on Software Supply Chain Offensive Research and Ecosystem Defenses*. Los Angeles CA USA: ACM, 2022, pp. 51–61.

[6] A. Alageel and S. Maffeis, "Hawk-Eye: Holistic detection of APT command and control domains," in *Proceedings of the 36th Annual ACM Symposium on Applied Computing*. Virtual Event Republic of Korea: ACM, 2021, pp. 1664–1673.

[7] A. A. Alsaheel, Y. Nan, S. Ma, L. Yu, G. Walkup, Z. B. Celik, X. Zhang, and D. Xu, "Atlas: A sequence-based learning approach for attack investigation," in *USENIX Security Symposium*, 2021.

[8] B. Bhattarai and H. Huang, "SteinerLog: Prize Collecting the Audit Logs for Threat Hunting on Enterprise Network," in *Proceedings of the 2022 ACM on Asia Conference on Computer and Communications Security*. Nagasaki Japan: ACM, 2022, pp. 97–108.

[9] H. Irshad, G. Ciocarlie, A. Gehani, V. Yegneswaran, K. H. Lee, J. Patel, S. Jha, Y. Kwon, D. Xu, and X. Zhang, "TRACE: Enterprise-Wide Provenance Tracking for Real-Time APT Detection," *IEEE Transactions on Information Forensics and Security*, vol. 16, pp. 4363–4376, 2021.

[10] W. U. Hassan, A. Bates, and D. Marino, "Tactical Provenance Analysis for Endpoint Detection and Response Systems," in *2020 IEEE Symposium on Security and Privacy (SP)*. San Francisco, CA, USA: IEEE, 2020, pp. 1172–1189.

[11] T. Sobb, B. Turnbull, and N. Moustafa, "Supply Chain 4.0: A Survey of Cyber Security Challenges, Solutions and Future Directions," *Electronics*, vol. 9, no. 11, p. 1864, 2020.

[12] A. Yeboah-Ofori and C. Boachie, "Malware Attack Predictive Analytics in a Cyber Supply Chain Context Using Machine Learning," in *2019 International Conference on Cyber Security and Internet of Things (ICSIoT)*. Accra, Ghana: IEEE, 2019, pp. 66–73.

[13] P. Ladisa, H. Plate, M. Martinez, O. Barais, and S. E. Ponta, "Towards the Detection of Malicious Java Packages," in *Proceedings of the 2022 ACM Workshop on Software Supply Chain Offensive Research and Ecosystem Defenses*. Los Angeles CA USA: ACM, 2022, pp. 63–72.

[14] R. Duan, O. Alrawi, R. P. Kasturi, R. Elder, B. Saltaformaggio, and W. Lee, "Towards Measuring Supply Chain Attacks on Package Managers for Interpreted Languages," 2020.

[15] N. Vasilakis, A. Benetopoulos, S. Handa, A. Schoen, J. Shen, and M. C. Rinard, "Supply-Chain Vulnerability Elimination via Active Learning and Regeneration," in *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*. Virtual Event Republic of Korea: ACM, 2021, pp. 1755–1770.

[16] P. Ladisa, H. Plate, M. Martinez, and O. Barais, "Taxonomy of Attacks on Open-Source Software Supply Chains," 2022.

[17] F. Wilkens, F. Ortmann, S. Haas, M. Vallentin, and M. Fischer, "Multi-Stage Attack Detection via Kill Chain State Machines," in *Proceedings of the 3rd Workshop on Cyber-Security Arms Race*. Virtual Event Republic of Korea: ACM, 2021, pp. 13–24.

[18] A. Yeboah-Ofori and S. Islam, "Cyber Security Threat Modeling for Supply Chain Organizational Environments," *Future Internet*, vol. 11, no. 3, p. 63, 2019.

[19] T. Steffens, *Attribution of Advanced Persistent Threats: How to Identify the Actors Behind Cyber-Espionage*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2020.

[20] A. Alshamrani, S. Myneni, A. Chowdhary, and D. Huang, "A survey on advanced persistent threats: Techniques, solutions, challenges, and research opportunities," *IEEE Communications Surveys & Tutorials*, vol. 21, no. 2, pp. 1851–1877, 2019.

[21] H. Li, J. Wu, H. Xu, G. Li, and M. Guizani, "Explainable Intelligence-Driven Defense Mechanism against Advanced Persistent Threats: A Joint Edge Game and AI Approach," *IEEE Transactions on Dependable and Secure Computing*, pp. 1–1, 2021.

[22] S. Eggers, "A novel approach for analyzing the nuclear supply chain cyber-attack surface," *Nuclear Engineering and Technology*, vol. 53, no. 3, pp. 879–887, 2021.

[23] M. I. Response, "Dev-0537 criminal actor targeting organizations for data exfiltration and destruction," 2022, [Online: Accessed 21 June 2023]. [Online]. Available: https://www.microsoft.com/en-us/security/blog/2022/03/22/dev-0537-criminal-actor-targeting-organizations-for-data-exfiltration-and-destruction/

[24] S. Khandelwal, "Sophisticated 'tajmahal apt framework' remained undetected for 5 years," 2019, [Online: Accessed 12 Apr 2023]. [Online]. Available: https://thehackernews.com/2019/04/apt-malware-framework.html

[25] Kaspersky, "Equation group: The crown creator of cyber-espionage," 2021, [Online: Accessed 12 June 2023]. [Online]. Available: https://www.kaspersky.com/about/press-releases/2015_equation-group-the-crown-creator-of-cyber-espionage

[26] S. D. Luis Magisa, "New macos backdoor connected to oceanlotus surfaces," 2020, [Online: Accessed 12 June 2023]. [Online]. Available: https://www.trendmicro.com/en_us/research/20/k/new-macos-backdoor-connected-to-oceanlotus-surfaces.html

[27] A. Drozhzhin, "Russian-speaking cyber spies exploit satellites," 2015, [Online: Accessed 16 June 2023]. [Online]. Available: https://www.kaspersky.co.uk/blog/turla-apt-exploiting-satellites/6210/

[28] H. Zhao, H. Shu, and Y. Xing, "A Review on IoT Botnet," in *The 2nd International Conference on Computing and Data Science*. Stanford CA USA: ACM, 2021, pp. 1–7.

[29] P. Lanigan, R. Gandhi, and P. Narasimhan, "Sluice: Secure dissemination of code updates in sensor networks," in *26th IEEE International Conference on Distributed Computing Systems (ICDCS'06)*, 2006, pp. 53–53.

[30] L. Borzacchiello, E. Coppa, D. C. D'Elia, and C. Demetrescu, "Reconstructing C2 servers for remote access trojans with symbolic execution," in *Cyber Security Cryptography and Machine Learning*, S. Dolev, D. Hendler, S. Lodha, and M. Yung, Eds. Cham: Springer International Publishing, 2019, pp. 121–140.

[31] A. Sidhardhan, K. S, and J. M. Kannimoola, "Weaponizing real-world applications as c2 (command and control)," in *2023 International Conference on Innovative Data Communication Technologies and Application (ICIDCA)*, 2023, pp. 458–463.

[32] J. Allen, Z. Yang, M. Landen, R. Bhat, H. Grover, A. Chang, Y. Ji, R. Perdisci, and W. Lee, "Mnemosyne: An Effective and Efficient Postmortem Watering Hole Attack Investigation System," in *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*. Virtual Event USA: ACM, 2020, pp. 787–802.

[33] Z. Gittins and M. Soltys, "Malware persistence mechanisms," *Procedia Computer Science*, vol. 176, pp. 88–97, 2020.

[34] A. Sharma, B. B. Gupta, A. K. Singh, and V. Saraswat, "Orchestration of APT malware evasive manoeuvers employed for eluding anti-virus and sandbox defense," *Computers & Security*, vol. 115, p. 102627, 2022.

[35] S. Beomju, P. Minhuck, J. Sanghoon, S. Hyoungmin, K. Gapjin, and K. Changdon, "Spoofing attack results determination in code domain using a spoofing process equation," *Sensors*, vol. 19, no. 2, pp. 293–, 2019.

[36] P. Lamprakis, R. Dargenio, D. Gugelmann, V. Lenders, M. Happe, and L. Vanbever, "Unsupervised detection of APT C&C channels using web request graphs," in *Detection of Intrusions and Malware, and Vulnerability Assessment*, M. Polychronakis and M. Meier, Eds. Cham: Springer International Publishing, 2017, pp. 366–387.

[37] S. Sibi Chakkaravarthy, D. Sangeetha, and V. Vaidehi, "A Survey on malware analysis and mitigation techniques," *Computer Science Review*, vol. 32, pp. 1–23, 2019.

[38] N. Moustafa, "Designing an online and reliable statistical anomaly detection framework for dealing with large high-speed network traffic," 2017. [Online]. Available: https://api.semanticscholar.org/CorpusID:46202299

[39] P. Shrivastava, M. S. Jamal, and K. Kataoka, "Evilscout: Detection and mitigation of evil twin attack in sdn enabled wifi," *IEEE Transactions on Network and Service Management*, vol. 17, no. 1, pp. 89–102, 2020.

[40] I. Dmitry, "Managing Risks in Supply Chains with Digital Twins and Simulation," [Online: Accessed 7 June 2023]. [Online]. Available: https://scms-summit.com/media/9417/managing-risks-in-supply-chains-with-digital-twins-and-simulation.pdf

[41] K. R. Ludvigsen, S. Nagaraja, and A. Daly, "Preventing or Mitigating Adversarial Supply Chain Attacks: A Legal Analysis," in *Proceedings of the 2022 ACM Workshop on Software Supply Chain Offensive Research and Ecosystem Defenses*. Los Angeles CA USA: ACM, 2022, pp. 25–34.

[42] C. Tools, "Cybersecurity framework v1.1," 2018, [Online: Accessed 12 July 2023]. [Online]. Available: https://csf.tools/reference/nist-cybersecurity-framework/v1-1/

[43] K. Stouffer, T. Zimmerman, C. Tang, J. Lubell, J. Cichonski, M. Pease, and J. McCarthy, "Cybersecurity Framework Version 1.1 Manufacturing Profile," National Institute of Standards and Technology, Tech. Rep., 2020.

[44] M. Prucková, "Supermicro supply chain breach (since 2010)," 2021, [Online: Accessed 1 July 2023]. [Online]. Available: https://cyberlaw.ccdcoe.org/wiki/SuperMicro_supply_chain_breach_(since_2010)

[45] picussecurity, "Tactics, techniques, and procedures (ttps) used by hafnium to target microsoft exchange servers," 2021, [Online: Accessed 5 July 2023]. [Online]. Available: https://www.picussecurity.com/resource/blog/ttps-hafnium-microsoft-exchange-servers

[46] V. Craciun, P. A. Felber, A. Mogage, E. Onica, and R. Pires, "Malware in the SGX supply chain: Be careful when signing enclaves!" *IEEE Transactions on Dependable and Secure Computing*, pp. 1–1, 2020.

[47] H. Al-Rushdan, M. Shurman, S. H. Alnabelsi, and Q. Althebyan, "Zero-day attack detection and prevention in software-defined networks," in *2019 International Arab Conference on Information Technology (ACIT)*, 2019, pp. 278–282.

[48] O. Pieczul and S. N. Foley, "Runtime detection of zero-day vulnerability exploits in contemporary software systems," in *Data and Applications Security and Privacy XXX*, S. Ranise and V. Swarup, Eds. Cham: Springer International Publishing, 2016, pp. 347–363.

[49] M. Fourné, D. Wermke, W. Enck, S. Fahl, and Y. Acar, "It's like flossing your teeth: On the importance and challenges of reproducible builds for software supply chain security," in *2023 2023 IEEE Symposium on Security and Privacy (SP) (SP)*. Los Alamitos, CA, USA: IEEE Computer Society, may 2023, pp. 1527–1544. [Online]. Available: https://doi.ieeecomputersociety.org/10.1109/SP46215.2023.00187

[50] D. L. Vu, I. Pashchenko, F. Massacci, H. Plate, and A. Sabetta, "Towards Using Source Code Repositories to Identify Software Supply Chain Attacks," in *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*. Virtual Event USA: ACM, 2020, pp. 2093–2095.

[51] RyotaK, "Remote code excution in cdnjs of cloudflare," 2021, [Online: Accessed 14 Mar 2023]. [Online]. Available: https://blog.ryotak.net/post/cdnjs-remote-code-execution-en/

[52] ——, "Remote code execution in homebrew by compromising the official cask repository," 2021, [Online: Accessed 14 Mar 2023]. [Online]. Available: https://blog.ryotak.net/post/homebrew-security-incident-en/

[53] ——, "Tampering with arbitrary packages in @types scope of npm," 2021, [Online: Accessed 14 Mar 2023]. [Online]. Available: https://blog.ryotak.net/post/definitelytyped-tamper-with-arbitrary-packages-en/

[54] ——, "Potential remote code execution in pypi," 2021, [Online: Accessed 14 Mar 2023]. [Online]. Available: https://blog.ryotak.net/post/pypi-potential-remote-code-execution-en/

[55] M. Ohm, H. Plate, A. Sykosch, and M. Meier, "Backstabber's Knife Collection: A Review of Open Source Software Supply Chain Attacks," 2020.

[56] J. Williams, "What you need to know about the solarwinds supply-chain attack — sans institute," 2020, [Online: Accessed 14 Mar 2023]. [Online]. Available: https://www.sans.org/blog/what-you-need-to-know-about-the-solarwinds-supply-chain-attack/

[57] FireEye, "Highly evasive attacker leverages solarwinds supply chain to compromise multiple global victims with sunburst backdoor," 2020, [Online: Accessed 16 Mar 2023]. [Online]. Available: https://www.mandiant.com/resources/blog/evasive-attacker-leverages-solarwinds-supply-chain-compromises-with-sunburst-backdoor

[58] D. Wermke, J. H. Klemmer, N. Wöhler, J. Schmüser, H. S. Ramulu, Y. Acar, and S. Fahl, "Always contribute back: A qualitative study on security challenges of the open source supply chain," in *2023 2023 IEEE Symposium on Security and Privacy (SP) (SP)*. Los Alamitos, CA, USA: IEEE Computer Society, may 2023, pp. 1545–1560. [Online]. Available: https://doi.ieeecomputersociety.org/10.1109/SP46215.2023.00191

[59] W. Jiang, N. Synovic, R. Sethi, A. Indarapu, M. Hyatt, T. R. Schorlemmer, G. K. Thiruvathukal, and J. C. Davis, "An Empirical Study of Artifacts and Security Risks in the Pre-trained Model Supply Chain," in *Proceedings of the 2022 ACM Workshop on Software Supply Chain Offensive Research and Ecosystem Defenses*. Los Angeles CA USA: ACM, 2022, pp. 105–114.

[60] K. Basu, S. M. Saeed, C. Pilato, M. Ashraf, M. T. Nabeel, K. Chakrabarty, and R. Karri, "CAD-Base: An Attack Vector into the Electronics Supply Chain," *ACM Transactions on Design Automation of Electronic Systems*, vol. 24, no. 4, pp. 1–30, 2019.

[61] M. Hildebrandt, K. Lamshöft, J. Dittmann, T. Neubert, and C. Vielhauer, "Information Hiding in Industrial Control Systems: An OPC UA based Supply Chain Attack and its Detection," in *Proceedings of the 2020 ACM Workshop on Information Hiding and Multimedia Security*. Denver CO USA: ACM, 2020, pp. 115–120.

[62] X. Yi, Y. Fang, D. Wu, and L. Jiang, "BlockScope: Detecting and Investigating Propagated Vulnerabilities in Forked Blockchain Projects," in *Proceedings 2023 Network and Distributed System Security Symposium*. San Diego, CA, USA: Internet Society, 2023.

[63] T. Kim, N. Park, J. Hong, and S.-W. Kim, "Phishing URL Detection: A Network-based Approach Robust to Evasion," in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*. Los Angeles CA USA: ACM, 2022, pp. 1769–1782.

[64] S. Das Guptta, K. T. Shahriar, H. Alqahtani, D. Alsalman, and I. H. Sarker, "Modeling Hybrid Feature-Based Phishing Websites Detection Using Machine Learning Techniques," *Annals of Data Science*, 2022.

[65] C. Cimpanu, "Eset discovers a rare apt that stayed undetected for nine years," 2020, [Online: Accessed 7 Apr 2023]. [Online]. Available: https://www.zdnet.com/article/eset-discovers-a-rare-apt-that-stayed-undetected-for-nine-years/

[66] F. L. Matthieu Faou, "Xdspy: Stealing government secrets since 2011," 2020, [Online: Accessed 3 June 2023]. [Online]. Available: https://vblocalhost.com/uploads/VB2020-Faou-Labelle.pdf

[67] T. Hegel, "Modifiedelephant apt and a decade of fabricating evidence," 2022, [Online: Accessed 10 Apr 2023]. [Online]. Available: https://www.sentinelone.com/labs/modifiedelephant-apt-and-a-decade-of-fabricating-evidence/

[68] K. T. Francis Guibernau, "Emulating recent activity from the russian adversary nobelium / apt29," 2023, [Online: Accessed 29 June 2023]. [Online]. Available: https://www.attackiq.com/2023/05/04/emulating-nobelium-apt29/

[69] Socradar, "Apt profile: Cozy bear / apt29," 2023, [Online: Accessed 5 June 2023]. [Online]. Available: https://socradar.io/apt-profile-cozy-bear-apt29/

[70] P. Shoshin, "Another taj mahal (between tokyo and yokohama)," 2019, [Online: Accessed 1 June 2023]. [Online]. Available: https://www.kaspersky.com/blog/taj-mahal-apt/26370/

[71] GReAT, "Sofacy apt hits high profile targets with updated toolset," 2015, [Online: Accessed 8 June 2023]. [Online]. Available: https:

//securelist.com/sofacy-apt-hits-high-profile-targets-with-updated-toolset/72924/

[72] D. MCWHORTER, "Apt28: A window into russia's cyber espionage operations?" 2014, [Online: Accessed 8 June 2023]. [Online]. Available: https://www.mandiant.com/resources/blog/apt28-a-window-into-russias-cyber-espionage-operations

[73] Kaspersky, "Advanced persistent threat actor lazarus attacks defense industry, develops supply chain attack capabilities," 2021, [Online: Accessed 8 June 2023]. [Online]. Available: https://www.kaspersky.com/about/press-releases/2021_advanced-persistent-threat-actor-lazarus-attacks-defense-industry-develops-supply-chain-attack-capabilities

[74] F. Guibernau, "Response to lazarus' 3cx supply chain compromise," 2023, [Online: Accessed 29 June 2023]. [Online]. Available: https://www.attackiq.com/2023/04/14/response-to-lazarus-3cx-supply-chain-compromise/

[75] Mandiant, "Equation group: The crown creator of cyber-espionage," 2021, [Online: Accessed 12 June 2023]. [Online]. Available: https://www.mandiant.com/sites/default/files/2021-09/mandiant-apt1-report.pdf

[76] S. Doherty, "Hidden lynx – professional hackers for hire," 2013, [Online: Accessed 12 June 2023]. [Online]. Available: https://www.wired.com/images_blogs/threatlevel/2013/09/hidden_lynx_final.pdf

[77] Kaspersky, "Kaspersky lab analyzes active cyberespionage campaign targeting online gaming companies worldwide," 2013, [Online: Accessed 12 June 2023]. [Online]. Available: https://www.kaspersky.com/about/press-releases/2013_kaspersky-lab-analyzes-active-cyberespionage-campaign-targeting-online-gaming-companies-worldwide

[78] J. Fokker, "Suspected darkhotel apt activity update," 2022, [Online: Accessed 12 June 2023]. [Online]. Available: https://www.trellix.com/en-us/about/newsroom/stories/research/suspected-darkhotel-apt-activity-update.html

[79] M. R. Jordan Robertson, "The long hack: How china exploited a u.s. tech supplier," 2021, [Online: Accessed 1 July 2023]. [Online]. Available: https://www.bloomberg.com/features/2021-supermicro/

[80] C. Xiong, T. Zhu, W. Dong, L. Ruan, R. Yang, Y. Cheng, Y. Chen, S. Cheng, and X. Chen, "A Practical Real-Time APT Detection System With High Accuracy and Efficiency," *IEEE Transactions on Dependable and Secure Computing*, vol. 19, no. 1, pp. 551–565, 2022.

[81] D. Zhao, H. Li, X. Sun, and Y. Tang, "DOLPHIN: Phonics based Detection of DGA Domain Names," in *2021 IEEE Global Communications Conference (GLOBECOM)*. Madrid, Spain: IEEE, 2021, pp. 01–06.

[82] S. Siby, M. Juarez, C. Diaz, N. Vallina-Rodriguez, and C. Troncoso, "Encrypted DNS –> Privacy? A Traffic Analysis Perspective," in *Proceedings 2020 Network and Distributed System Security Symposium*. San Diego, CA: Internet Society, 2020.

[83] P. Jeitner and H. Shulman, "Injection attacks reloaded: Tunnelling malicious payloads over DNS," in *30th USENIX Security Symposium (USENIX Security 21)*. USENIX Association, Aug. 2021, pp. 3165–3182. [Online]. Available: https://www.usenix.org/conference/usenixsecurity21/presentation/jeitner

[84] B. Soman, "Death to the ioc, what's next in threat intelligence," 2019, black Hat. [Online]. Available: https://i.blackhat.com/USA-19/Thursday/us-19-Soman-Death-To-The-IOC-Whats-Next-In-Threat-Intelligence.pdf

[85] C. Gao, X. Zhang, and H. Liu, "Data and knowledge-driven named entity recognition for cyber security," *Cybersecurity*, vol. 4, no. 1, p. 9, 2021.

[86] L. Lab, "What is wannacry ransomware?" 2017, [Online: Accessed 17 July 2023]. [Online]. Available: https://www.kaspersky.com/resource-center/threats/ransomware-wannacry

[87] P. Dodia, M. AlSabah, O. Alrawi, and T. Wang, "Exposing the Rat in the Tunnel: Using Traffic Analysis for Tor-based Malware Detection," in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*. Los Angeles CA USA: ACM, 2022, pp. 875–889.

[88] S. White, "Tajmahal advanced persistent threat," 2019, [Online: Accessed 5 June 2023]. [Online]. Available: https://www.infoblox.com/wp-content/uploads/threat-intelligence-report-tajmajal-advanced-persistent-threat.pdf

[89] O. VLCEK, "Recent findings from ccleaner apt investigation reveal that attackers entered the piriform network via teamviewer," 2018, [Online: Accessed 4 June 2023]. [Online]. Available: https://blog.avast.com/update-ccleaner-attackers-entered-via-teamviewer

[90] K. Tummala, "Ccleaner apt attack: A technical look inside," 2018, [Online: Accessed 4 June 2023]. [Online]. Available: https://www.cisoplatform.com/profiles/blogs/ccleaner-apt-attack-a-technical-look-inside

[91] K. Lab, "Operation shadowhammer: new supply chain attack threatens hundreds of thousands of users worldwide," 2019, [Online: Accessed 4 June 2023]. [Online]. Available: https://www.kaspersky.com/about/press-releases/2019_operation-shadowhammer-new-supply-chain-attack

[92] S. Gatlan, "Asus live update infected with backdoor in supply chain attack," 2019, [Online: Accessed 4 June 2023]. [Online]. Available: https://www.bleepingcomputer.com/news/security/asus-live-update-infected-with-backdoor-in-supply-chain-attack/

[93] S. H. Karan Sood, "Notpetya technical analysis – a triple threat: File encryption, mft encryption, credential theft," 2017, [Online: Accessed 3 July 2023]. [Online]. Available: https://www.crowdstrike.com/blog/petrwrap-ransomware-technical-analysis-triple-threat-file-encryption-mft-encryption-credential-theft/

[94] J. Allen, "Kaseya vsa ransomware attack explained," 2023, [Online: Accessed 6 June 2023]. [Online]. Available: https://purplesec.us/kaseya-ransomware-attack-explained/

[95] S. Ozarslan, "Ttps used by revil sodinokibi ransomware gang in kaseya msp supply-chain attack," 2021, [Online: Accessed 5 July 2023]. [Online]. Available: https://www.picussecurity.com/resource/blog/revil-sodinokibi-ransomware-kaseya-vsa-msp-supply-chain-attack

[96] CISA, "Exploitation of accellion file transfer appliance," 2021, [Online: Accessed 6 June 2023]. [Online]. Available: https://www.cisa.gov/news-events/cybersecurity-advisories/aa21-055a

[97] A. HOPE, "Codecov supply chain attack remained undetected for months and potentially affected major companies including google, ibm, hp, and others," 2021, [Online: Accessed 8 June 2023]. [Online]. Available: https://www.cpomagazine.com/cyber-security/codecov-supply-chain-attack-remained-undetected-for-months-and-potentially-affected-major-companies-including-google-ibm-hp-and-others/

[98] A. Sharma, "What you need to know about the codecov incident: A supply chain attack gone undetected for 2 months," 2021, [Online: Accessed 8 June 2023]. [Online]. Available: https://blog.sonatype.com/what-you-need-to-know-about-the-codecov-incident-a-supply-chain-attack-gone-undetected-for-2-months

[99] M. . Security, "Hafnium targeting exchange servers with 0-day exploits," 2021, [Online: Accessed 8 June 2023]. [Online]. Available: https://www.microsoft.com/en-us/security/blog/2021/03/02/hafnium-targeting-exchange-servers/

[100] swimlane, "A python package to interact with the mitre att&ck framework," [Online: Accessed 4 July 2023]. [Online]. Available: https://github.com/swimlane/pyattck

[101] M. Bachmann, "A python package to interact with the mitre att&ck framework," 2023, [Online: Accessed 6 July 2023]. [Online]. Available: https://github.com/maxbachmann/RapidFuzz

[102] Z. Yang, B. Xu, J. M. Zhang, H. J. Kang, J. Shi, J. He, and D. Lo, "Stealthy Backdoor Attack for Code Models," 2023.

[103] R. Alkhadra, J. Abuzaid, M. AlShammari, and N. Mohammad, "Solar Winds Hack: In-Depth Analysis and Countermeasures," in *2021 12th International Conference on Computing Communication and Networking Technologies (ICCCNT)*. Kharagpur, India: IEEE, 2021, pp. 1–7.

[104] J. Martínez and J. M. Durán, "Software Supply Chain Attacks, a Threat to Global Cybersecurity: SolarWinds' Case Study," *International Journal of Safety and Security Engineering*, vol. 11, no. 5, pp. 537–545, 2021.

[105] S. Peisert, B. Schneier, H. Okhravi, F. Massacci, T. Benzel, C. Landwehr, M. Mannan, J. Mirkovic, A. Prakash, and J. B. Michael, "Perspectives on the SolarWinds Incident," *IEEE Security & Privacy*, vol. 19, no. 2, pp. 7–13, 2021.

[106] F. Nicolas, O. M. Liam, and C. Eric, "W32.stuxnet dossier," 2011, [Online: Accessed 1 June 2023]. [Online]. Available: https://scms-summit.com/media/9417/managing-risks-in-supply-chains-with-digital-twins-and-simulation.pdf

[107] R. Kumar, R. Kela, S. Singh, and R. Trujillo-Rasua, "APT attacks on industrial control systems: A tale of three incidents," *International Journal of Critical Infrastructure Protection*, vol. 37, no. C, 2022. [Online]. Available: https://ideas.repec.org/a/eee/ijocip/v37y2022ics1874548222000129.html

[108] S. Wrycza and J. Maślankowski, Eds., *Information Systems: Research, Development, Applications, Education: 12th SIGSAND/PLAIS EuroSymposium 2019, Gdansk, Poland, September 19, 2019, Proceedings*, ser. Lecture Notes in Business Information Processing. Cham: Springer International Publishing, 2019, vol. 359.

[109] G. Ho, M. Dhiman, D. Akhawe, V. Paxson, S. Savage, G. M. Voelker, and D. Wagner, "Hopper: Modeling and Detecting Lateral Movement (Extended Report)," 2021.

[110] I. J. King and H. H. Huang, "Euler: Detecting Network Lateral Movement via Scalable Temporal Link Prediction," *ACM Transactions on Privacy and Security*, p. 3588771, 2023.

[111] W.-N. Niu, J. Xie, X.-S. Zhang, C. Wang, X.-Q. Li, R.-D. Chen, and X.-L. Liu, "HTTP-Based APT Malware Infection Detection Using URL Correlation Analysis," *Security and Communication Networks*, vol. 2021, pp. 1–12, 2021.

[112] G. Yan, Q. Li, D. Guo, and X. Meng, "Discovering Suspicious APT Behaviors by Analyzing DNS Activities," *Sensors*, vol. 20, no. 3, p. 731, 2020.

[113] J. Yang, Q. Zhang, X. Jiang, S. Chen, and F. Yang, "Poirot: Causal Correlation Aided Semantic Analysis for Advanced Persistent Threat Detection," *IEEE Transactions on Dependable and Secure Computing*, vol. 19, no. 5, pp. 3546–3563, 2022.

[114] A. Imran and R. Agrawal, "Data Provenance," in *Encyclopedia of Big Data*, L. A. Schintler and C. L. McNeely, Eds. Cham: Springer International Publishing, 2017, pp. 1–4.

[115] W. Ametepe, C. Wang, S. K. Ocansey, X. Li, and F. Hussain, "Data provenance collection and security in a distributed environment: A survey," *International Journal of Computers and Applications*, vol. 43, no. 1, pp. 11–25, 2021.

[116] R. Shanmugam, "Elements of causal inference: Foundations and learning algorithms," *Journal of Statistical Computation and Simulation*, vol. 88, no. 16, pp. 3248–3248, 2018.

[117] A. R. Nogueira, A. Pugnana, S. Ruggieri, D. Pedreschi, and J. Gama, "Methods and tools for causal discovery and causal inference," *WIREs Data Mining and Knowledge Discovery*, vol. 12, no. 2, 2022.

[118] X. Shen, S. Ma, P. Vemuri, G. Simon, and the Alzheimer's Disease Neuroimaging Initiative, "Challenges and Opportunities with Causal Discovery Algorithms: Application to Alzheimer's Pathophysiology," *Scientific Reports*, vol. 10, no. 1, p. 2975, 2020.

[119] C. Glymour, K. Zhang, and P. Spirtes, "Review of Causal Discovery Methods Based on Graphical Models," *Frontiers in Genetics*, vol. 10, p. 524, 2019.

[120] B. Huang, K. Zhang, J. Zhang, J. Ramsey, R. Sanchez-Romero, C. Glymour, and B. Schölkopf, "Causal Discovery from Heterogeneous/Nonstationary Data with Independent Changes," 2020.

[121] S. Shimizu, P. O. Hoyer, A. Hyv, rinen, and A. Kerminen, "A linear non-gaussian acyclic model for causal discovery," *Journal of Machine Learning Research*, vol. 7, no. 72, pp. 2003–2030, 2006. [Online]. Available: http://jmlr.org/papers/v7/shimizu06a.html

[122] K. Zhang and A. Hyvärinen, "On the identifiability of the post-nonlinear causal model," in *Proceedings of the Twenty-Fifth Conference on Uncertainty in Artificial Intelligence*, ser. UAI '09. Arlington, Virginia, USA: AUAI Press, 2009, p. 647–655.

[123] P. O. Hoyer, D. Janzing, J. Mooij, J. Peters, and B. Schölkopf, "Nonlinear causal discovery with additive noise models," in *Proceedings of the 21st International Conference on Neural Information Processing Systems*, ser. NIPS'08. Red Hook, NY, USA: Curran Associates Inc., 2008, p. 689–696.

[124] F. Xie, R. Cai, B. Huang, C. Glymour, Z. Hao, and K. Zhang, "Generalized Independent Noise Condition for Estimating Latent Variable Causal Graphs," 2020.

[125] F. Xie, B. Huang, Z. Chen, Y. He, Z. Geng, and K. Zhang, "Identification of linear non-Gaussian latent hierarchical structure," in *Proceedings of the 39th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, K. Chaudhuri, S. Jegelka, L. Song, C. Szepesvari, G. Niu, and S. Sabato, Eds., vol. 162. PMLR, 17–23 Jul 2022, pp. 24 370–24 387. [Online]. Available: https://proceedings.mlr.press/v162/xie22a.html

[126] J. M. Rohrer, "Thinking Clearly About Correlations and Causation: Graphical Causal Models for Observational Data," *Advances in Methods and Practices in Psychological Science*, pp. 27–42, 2018.

[127] D. Malinsky and D. Danks, "Causal discovery algorithms: A practical guide," *Philosophy Compass*, vol. 13, no. 1, p. e12470, 2018.

[128] D. H. Xiong, "Association analysis: Basic concepts and algorithms," 2005. [Online]. Available: https://api.semanticscholar.org/CorpusID:46666286

[129] B. Wang, T. Zhou, S. Li, Y. Cao, and N. Gong, "GraphTrack: A Graph-based Cross-Device Tracking Framework," in *Proceedings of the 2022 ACM on Asia Conference on Computer and Communications Security*. Nagasaki Japan: ACM, 2022, pp. 82–96.

[130] Z. A. El Houda, A. S. Hafid, and L. Khoukhi, "A Novel Machine Learning Framework for Advanced Attack Detection using SDN," in *2021 IEEE Global Communications Conference (GLOBECOM)*. Madrid, Spain: IEEE, 2021, pp. 1–6.

[131] Y. Shen, M. Simsek, B. Kantarci, H. T. Mouftah, M. Bagheri, and P. Djukic, "Prior Knowledge based Advanced Persistent Threats Detection for IoT in a Realistic Benchmark," in *GLOBECOM 2022 - 2022 IEEE Global Communications Conference*. Rio de Janeiro, Brazil: IEEE, 2022, pp. 3551–3556.

[132] M. Naseri, Y. Han, E. Mariconti, Y. Shen, G. Stringhini, and E. D. Cristofaro, "Cerberus: Exploring federated prediction of security events," 2022.

[133] P. Gohel, P. Singh, and M. Mohanty, "Explainable AI: Current status and future directions," 2021.

[134] K. Sood, "Protecting the software supply chain: Deep insights into the ccleaner backdoor," 2017, [Online: Accessed 3 July 2023]. [Online]. Available: https://www.crowdstrike.com/blog/protecting-software-supply-chain-deep-insights-ccleaner-backdoor/

[135] L. Labs, "Notpetya technical analysis," 2017, [Online: Accessed 3 July 2023]. [Online]. Available: https://gallery.logrhythm.com/threat-intelligence-reports/notpetya-technical-analysis-logrhythm-labs-threat-intelligence-report.pdf

[136] J. W. Francis Guibernau, Ken Towne, "Oilrig attack graphs: Emulating the iranian threat actor's global campaigns," 2022, [Online: Accessed 30 June 2023]. [Online]. Available: https://www.attackiq.com/2022/07/11/oilrig-attack-graphs-emulating-the-iranian-threat-actors-global-campaigns/

[137] S. M. Milajerdi, R. Gjomemo, B. Eshete, R. Sekar, and V. Venkatakrishnan, "Holmes: Real-time apt detection through correlation of suspicious information flows," in *2019 IEEE Symposium on Security and Privacy (SP)*, 2019, pp. 1137–1152.

[138] F. P. Miller, A. F. Vandome, and J. McBrewster, *Levenshtein Distance: Information Theory, Computer Science, String (Computer Science), String Metric, Damerau?Levenshtein Distance, Spell Checker, Hamming Distance*. Alpha Press, 2009.

[139] C. Paulsen, J. Boyens, J. Ng, K. Winkler, and J. Gimbi, "Impact analysis tool for interdependent cyber supply chain risks," National Institute of Standards and Technology, Gaithersburg, MD, Tech. Rep. NIST IR 8272, 2020.

[140] L.-X. Yang, P. Li, X. Yang, and Y. Y. Tang, "A Risk Management Approach to Defending Against the Advanced Persistent Threat," *IEEE Transactions on Dependable and Secure Computing*, vol. 17, no. 6, pp. 1163–1172, 2020.

[141] A. Berady, M. Jaume, V. V. T. Tong, and G. Guette, "From ttp to ioc: Advanced persistent graphs for threat hunting," *IEEE Transactions on Network and Service Management*, vol. 18, no. 2, pp. 1321–1333, 2021.

[142] E. Bandara, S. Shetty, A. Rahman, and R. Mukkamala, "Let'sTrace — Blockchain, Federated Learning and TUF/In-ToTo Enabled Cyber Supply Chain Provenance Platform," in *MILCOM 2021 - 2021 IEEE Military Communications Conference (MILCOM)*. San Diego, CA, USA: IEEE, 2021, pp. 470–476.